HACKATHON SARD · EXPO ASI 2026

Arquitectura y plan de desarrollo de la app móvil SARD

Sistema Agéntico de Respuesta y Resiliencia ante Desastres: análisis del enunciado, arquitectura recomendada, protocolo, agentes, pruebas y hoja de ruta.

Evento Universidad Nacional de Colombia | Sede Manizales

Departamento Informática y Computación

Repositorio hackathon-asi

Versión 0.2 · desarrollo individual con Flutter y Dart · 8 de noviembre de 2026

LAS DECISIONES DE DISEÑO SON PROPUESTAS · SUPUESTOS MARCADOS

Documento derivado del enunciado «HackathonExpoASI2026.pdf» (12 páginas). Las cifras de tamaño de trama provienen de la calculadora de referencia `docs/src/tools/frame_size.py`; las metas de las pruebas son sugerencias a calibrar.

Índice de contenido

1 Resumen ejecutivo 4
- 1.1 Las ocho decisiones que se recomiendan 4
- 1.2 Pila tecnológica propuesta 5

2 Análisis del enunciado 6
- 2.1 Qué plantea el enunciado 6
- 2.2 Requisitos extraídos 6
- 2.3 Lo que el enunciado no dice 7
- 2.4 Implicaciones de diseño que se desprenden del texto 8

3 Arquitectura recomendada 9
- 3.1 Principios de diseño 9
- 3.2 Vista general del sistema 10
- 3.3 Por qué la app móvil no es todo el sistema 11
- 3.4 Pila tecnológica y alternativas 11
- 3.5 El recorrido de integración de punta a punta 11

4 Matriz de trazabilidad 13
- 4.1 Las diez preguntas de los escenarios 13
- 4.2 Los diez objetivos específicos 13
- 4.3 Retos, entregables y recorrido de integración 14

5 La app móvil 15
- 5.1 Qué hace y qué no hace 15
- 5.2 Arquitectura interna 15
- 5.3 Experiencia: reportar en tres toques 16
- 5.4 Datos locales y estados de entrega 17
- 5.5 Planificador de envío 18
- 5.6 Ahorro de batería 19
- 5.7 Elección del framework móvil 19
- 5.8 Riesgos específicos del móvil 20

6 Nodos supervivientes y tejido de agentes 21
- 6.1 El nodo superviviente 21
- 6.2 Sincronización, pérdida de nodo y recuperación 22
- 6.3 El tejido de agentes 23
- 6.4 Fichas de los agentes 24
- 6.5 Integración con modelos de lenguaje 27
- 6.6 Deduplicación, independencia de fuentes e incertidumbre 27
- 6.7 El tablero del coordinador 29

7 Protocolo de comunicación SARD-WIRE v0 32
- 7.1 Objetivos y supuestos del canal 32
- 7.2 Formato de la trama 32
- 7.3 Tipos de mensaje 33
- 7.4 Fiabilidad: confirmaciones, reintentos y fragmentación 34
- 7.5 Deduplicación en el transporte y revisiones 34
- 7.6 Seguridad, integridad y abuso 35
- 7.7 Qué se midió 35
- 7.8 Canal del despachador (fuera de la trama) 35

8 IA responsable y seguridad **37**

9 Estrategia de pruebas y métricas **39**

9.1 El banco de pruebas 39
9.2 Catálogo de pruebas 40
9.3 Métricas 42
9.4 Informe de resultados (plantilla del entregable 6) 44

10 Plan de desarrollo **45**

10.1 Estructura del repositorio 45
10.2 Hoja de ruta por bloques 46
10.3 Si el tiempo es corto: qué mantener y qué simplificar 47
10.4 Roles 47
10.5 Riesgos del proyecto 47
10.6 Guion de la demostración 48
10.7 Próximos pasos inmediatos 48

A Glosario **50**

B Decisiones de arquitectura (resumen) **51**

1 Resumen ejecutivo

El enunciado de la Hackathon SARD plantea un sismo de magnitud 7,4 seguido de un apagón generalizado y de la caída en cascada de antenas, centros de datos y proveedores de internet. El desafío no es solamente recoger información: es **decidir qué hacer con ella** cuando llega parcial, duplicada y contradictoria, y cuando el sistema que la procesa puede desaparecer en cualquier momento. Este documento analiza el enunciado y propone una arquitectura concreta para una **aplicación móvil** que forma parte de un sistema mayor, junto con su protocolo, sus agentes, la estrategia de pruebas y un plan de desarrollo.

> **LA CONCLUSIÓN MÁS IMPORTANTE**
> El enunciado no evalúa una aplicación: evalúa **un recorrido completo de ocho pasos** (reporte ciudadano → almacenamiento local → transmisión intermitente → procesamiento por agentes → priorización → tablero → pérdida de un nodo → recuperación). La app móvil es la primera capa de ese recorrido y el cliente de su transporte, pero por sí sola no cumple el requisito de integración ni los siete entregables. Por eso la arquitectura propuesta cubre además los nodos supervivientes, el tejido de agentes, el tablero del coordinador y un banco de pruebas.

1.1 Las ocho decisiones que se recomiendan

1. **Offline-first y store-and-forward.** El reporte nace y se guarda en el teléfono (SQLite durable). La pantalla sólo dice «Guardado» después del *commit*. La app nunca depende de la red para ser útil.
2. **Reporte en tres toques y por revisiones.** Primero un SOS mínimo (tipo, ubicación, hora) que se encola de inmediato; los detalles son revisiones posteriores del mismo reporte. Así se minimiza la interacción y se evitan duplicados desde el origen.
3. **Trama binaria propia con identificador idempotente.** El SOS mínimo ocupa **33 bytes**, frente a 239 bytes de un JSON con claves descriptivas y a 104 bytes de un JSON ya compactado (medido con la calculadora del repositorio). El identificador del reporte (12 bytes) hace que reintentar sea siempre seguro.
4. **Sin nodo principal.** Cada nodo superviviente guarda un log de eventos que sólo crece; las vistas (cola priorizada, incidentes) se derivan de él. Los nodos se sincronizan por anti-entropía al reconectarse. La caída de un nodo no detiene al sistema.
5. **Agentes con respaldo determinista.** Los seis agentes del enunciado son suscriptores del log. El modelo de lenguaje mejora el triaje, la correlación y las recomendaciones, pero ninguna función esencial depende de él; el agente de resiliencia no lo usa.
6. **Autonomía limitada por construcción.** No existe ninguna herramienta de despacho al alcance de los agentes: sólo pueden *proponer*. Toda recomendación exige aprobación humana, cita sus fuentes y marca su incertidumbre.
7. **Dos niveles de deduplicación.** El transporte descarta reintentos por identificador; la correlación semántica agrupa reportes distintos del mismo suceso como una *vista* reversible que conserva el vínculo con cada observación original.
8. **Banco de pruebas de primera clase.** Un generador sintético con verdad de referencia, un simulador de canal (pérdida, duplicados, ventanas cortas) y un inyector de fallos permiten reproducir los escenarios con una semilla y producir los resultados del entregable 6.

1.2 Pila tecnológica propuesta

Pieza	Propuesta por defecto	Estado
App móvil	Flutter (Dart). Android (emulador) como plataforma principal; iOS en simulador como segunda	**Decidido** una persona que ya domina Flutter
Nodos y agentes	Dart puro (`dart run`), SQLite en modo WAL con `package:sqlite3`	**Validado** SQLite ante `kill -9` y recorrido H1 (ver `docs/resultados/h1.md`)
Tablero	Flutter Web, servido por cada nodo; comparte el paquete de protocolo	**Propuesta mía**
Modelo de lenguaje	API de Claude detrás de un puerto intercambiable, con respaldo de reglas y opción de modelo local	**Propuesta mía** depende de credenciales y presupuesto
Estructura del repositorio	Monorepo con *pub workspaces* de Dart: `apps/` y `packages/` (paquetes compartidos en Dart puro)	**Propuesta mía**

LO QUE EL ENUNCIADO NO DICE Y TODAVÍA FALTA CONFIRMAR

El documento no indica el plazo ni la duración de la hackathon, los criterios ni los pesos de evaluación, ni el proveedor de IA permitido. Ya están **confirmados** el equipo (una persona), el framework (Flutter) y los dispositivos de prueba (emulador de Android y simulador de iOS). El resto son **valores por defecto míos**, no requisitos; la sección §2.3 lista cada hueco con el supuesto adoptado. **Lo que más falta para dimensionar el recorte es el plazo disponible.**

2 Análisis del enunciado

Este capítulo extrae del documento «Hackathon SARD» lo que se debe construir y demostrar, separa lo que el texto exige de lo que sólo sugiere y deja explícito lo que no dice. Es la base de la matriz de trazabilidad del capítulo §4.

2.1 Qué plantea el enunciado

El escenario es un sismo M 7,4 en la región andina colombiana que desencadena una cascada de fallos: el Sistema Interconectado Nacional pierde estabilidad, las antenas móviles y los centros de datos quedan sin energía, los proveedores regionales de internet se desconectan y las comunicaciones entre organismos de socorro se fragmentan. La UNGRD, la Cruz Roja y los bomberos reciben información «parcial, duplicada, contradictoria y desactualizada».

El problema se formula como cuatro decisiones críticas que el sistema debe resolver: **urgencia** (qué requiere respuesta inmediata), **confiabilidad** (qué tan creíble es), **destino** (a dónde enviarlo) y **persistencia** (cómo conservarlo si el sistema que lo procesa desaparece). SARD se define como una capa de coordinación de inteligencia artificial agéntica distribuida entre los ciudadanos, los nodos de comunicación supervivientes y los equipos humanos de respuesta.

> **EL CRITERIO REAL DE ÉXITO**
> La sección de integración exige demostrar **«al menos un recorrido completo, desde que el ciudadano crea el reporte hasta que el sistema se recupera tras perder un nodo»**. Se permite usar datos sintéticos y canales simulados; no se exige una red de radio real ni acceso a infraestructura gubernamental. La evaluación, además, «no debe depender únicamente de la presentación final»: pide evidencia técnica reproducible y auditable.

2.2 Requisitos extraídos

Los dos escenarios obligatorios y sus diez preguntas

El enunciado formula cinco preguntas por escenario. Cada una se vuelve un requisito verificable y se rastrea en el capítulo §4.

Escenario 1 · El momento del apagón

Primeros minutos; baterías por debajo del 20 %; conectividad casi inexistente.

- **E1.1** ¿Cómo crear y guardar un reporte sin conexión?
- **E1.2** ¿Cómo enviarlo con el menor número de bytes?
- **E1.3** ¿Cómo evitar perderlo si la transmisión se interrumpe?
- **E1.4** ¿Cómo saber si es una emergencia inmediata?
- **E1.5** ¿Cómo evitar que un mismo incidente se registre decenas de veces?

Escenario 2 · El silencio digital

Pasadas 6 h; 95 % de los teléfonos apagados; despachadores en «oscuridad informativa».

- **E2.1** ¿Cómo continúa el sistema si el nodo principal se apaga sin aviso?
- **E2.2** ¿Cómo se sincronizan nodos que trabajaron aislados?
- **E2.3** ¿Cómo se combinan fragmentos sin inventar datos?
- **E2.4** ¿Cómo se identifica y administra la incertidumbre?
- **E2.5** ¿Cómo se entrega una lista corta de prioridades sin ocultar casos relevantes?

Los seis retos y su prioridad

Tabla 1. *Retos oficiales. Todos los equipos deben resolver el flujo completo y pueden profundizar en uno o varios de los retos 4 a 6.*

Reto	Nombre	Dimensión	Prioridad	Qué exige
1	El reporte que debe sobrevivir	Captura ciudadana	Obligatoria	Interfaz ultraligera que registra un incidente, lo guarda localmente y lo deja listo para transmitir sin conexión.
2	Comunicación en ráfagas	Transporte de datos	Obligatoria	Cola de mensajes pequeños que soporte pérdidas, interrupciones, reintentos y recepción desordenada.
3	El tejido de agentes	IA agéntica	Diferenciador	Agentes especializados que interpreten, validen, enriquezcan y prioricen, sin depender de un agente central.
4	La verdad entre fragmentos	Deduplicación, incertidumbre	Profundización	Detectar repetidos, relacionar observaciones y conservar las contradicciones que puedan cambiar la respuesta.
5	Sobrevivir a la caída	Resiliencia, sincronización	Profundización	Conservar los reportes y continuar las funciones esenciales cuando desaparece el nodo principal.
6	El tablero que salva tiempo	Coordinación humana	Profundización	Vista ordenada por prioridades, con evidencia, incertidumbre, ubicación disponible y siguiente acción sugerida.

Los siete entregables obligatorios

Tabla 2. *Entregables oficiales y capítulo de este documento que los respalda.*

N.º	Entregable	Contenido requerido por el enunciado	Dónde se cubre aquí
1	Prototipo funcional	Aplicación o servicio demostrable con el flujo mínimo de reporte, procesamiento y priorización.	§10 (bloques B1–B5)
2	Código fuente	Repositorio accesible al jurado, con instalación, dependencias y configuración para reproducir la prueba.	§10.1
3	Documento de arquitectura	Diagrama de componentes, distribución de agentes, flujos de mensajes, almacenamiento y dependencias.	§3, §5 y §6
4	Fichas de agentes	Objetivo, entradas, salidas, herramientas, memoria, límites de autonomía, manejo de errores y criterios de terminación.	§6.4
5	Protocolo de comunicación	Esquema del mensaje, reintentos, confirmaciones, deduplicación y estrategia de sincronización.	§7
6	Resultados de pruebas	Métricas, escenarios ejecutados, errores observados, limitaciones y resultados de las pruebas de estrés.	§9
7	Pitch y demostración	Presentación ejecutiva y demostración reproducible bajo al menos una condición adversa.	§10.6

2.3 Lo que el enunciado no dice

El documento es deliberadamente abierto: la arquitectura de referencia es «orientación común, no una solución obligatoria». Estos son los vacíos que afectan al diseño y el supuesto de trabajo adoptado. Ninguno se ha inventado como requisito.

Tabla 3. *Vacíos del enunciado y supuestos de trabajo. Todos son revisables.*

Vacío	Supuesto de trabajo	Si resulta distinto
Plazo y duración	Se desconoce. El plan usa porcentajes del tiempo disponible, no fechas.	Con poco tiempo, aplicar el recorte de §10.3.
Tamaño y perfil del equipo	**Confirmado:** una persona que domina Flutter.	El plan se recorta a la columna «Mantener» de §10.3; ver ADR-10 en el anexo B.
Plataforma móvil	**Confirmado:** emulador de Android como principal y simulador de iOS como segundo. Los emuladores no prueban batería ni radio reales.	El reintento en segundo plano de iOS es más débil: la demostración se hace con la app en primer plano.
Criterios y pesos de evaluación	Se asume peso relevante en resiliencia, evidencia reproducible y uso responsable de IA.	Reordenar qué retos de profundización se eligen.
Modelo de lenguaje y presupuesto	API de Claude disponible durante el desarrollo y la demo; el sistema funciona también sin ella.	Sin acceso: reglas deterministas o modelo local; el diseño ya lo contempla.
Datos de ejemplo	No hay conjunto de datos oficial; se generan sintéticos con verdad de referencia.	Si el jurado entrega datos, se cargan con el mismo formato del generador.
Escala objetivo	No se cuantifica el número de reportes, dispositivos ni nodos. Se propone una prueba de ráfaga de 10 000 reportes y de 3 a 5 nodos.	Ajustar la prueba T11; la arquitectura escala por nodo.
Canales reales	Se permiten canales simulados. HTTP, BLE/Nearby y SMS se dejan como implementaciones de una misma interfaz.	Si el jurado valora un canal real, implementar uno (la interfaz ya está definida).

2.4 Implicaciones de diseño que se desprenden del texto

Estas restricciones no son preferencias: están en el enunciado y condicionan la arquitectura.

1. **Dispositivo moribundo.** Baterías por debajo del 20 % y videos pesados drenando energía: la app debe ser mínima en CPU, pantalla y radio, sin medios.
2. **Conectividad casi inexistente.** El sistema no puede asumir una sesión: transmite en *ventanas* cortas y por ráfagas, con reintentos y recepción desordenada.
3. **Bytes como recurso escaso.** «El menor número posible de bytes» justifica un formato binario compacto y fragmentos que quepan en canales de baja capacidad.
4. **Duplicado no es lo mismo en transporte que en semántica.** Reenviar el mismo mensaje y reportar el mismo suceso desde dos teléfonos son problemas distintos, con soluciones distintas.
5. **Nunca borrar evidencia ni fusionar casos distintos.** La correlación debe ser una vista reversible con vínculo a cada observación original.
6. **Priorizar sin ocultar.** «La priorización no debe hacer desaparecer los casos de prioridad baja» y la lista corta no debe «ocultar casos relevantes».
7. **El agente de resiliencia no usa LLM.** Es una indicación explícita: la infraestructura no la decide un modelo de lenguaje.
8. **Autonomía acotada.** No inventar ubicación ni datos, no despachar ni asignar recursos críticos sin autorización.
9. **Descentralización.** Sin agente central único; reconstruir el estado desde nodos que estuvieron aislados.
10. **Reproducibilidad.** Pruebas de desconexión, pérdida y recuperación repetibles y auditables.

3 Arquitectura recomendada

La arquitectura sigue la referencia de cuatro capas del enunciado (ciudadana, nodos supervivientes, tejido de agentes y centro de coordinación humana), la concreta con decisiones técnicas y añade un componente que el enunciado deja implícito pero que sus entregables 6 y 7 requieren: un **banco de pruebas** reproducible.

3.1 Principios de diseño

1 · Lo local manda

La verdad de un reporte nace en el dispositivo y se guarda allí antes de cualquier intento de red. La red es una optimización, no un requisito.

2 · Los hechos no se borran

El sistema guarda observaciones y eventos que sólo se añaden. Las conclusiones (grupos, prioridades) son vistas derivables, revisables y reversibles.

3 · Reintentar es siempre seguro

Cada reporte y cada revisión tienen un identificador estable; procesarlos dos veces produce el mismo estado (idempotencia).

4 · Degradación escalonada

Sin Internet → sin modelo de lenguaje → sin pares → sólo el teléfono. En cada escalón se conserva una función útil.

5 · La IA propone, el humano decide

Los agentes no tienen capacidad de despachar. Toda acción requiere una aprobación registrada.

6 · Toda afirmación cita su evidencia

Cada salida de un agente enlaza los eventos de entrada; una cita que no exista en la entrada se rechaza.

7 · Medir antes de afirmar. Las propiedades de resiliencia, entrega y calidad se demuestran con pruebas reproducibles, no con la narración de la demostración.

3.2 Vista general del sistema

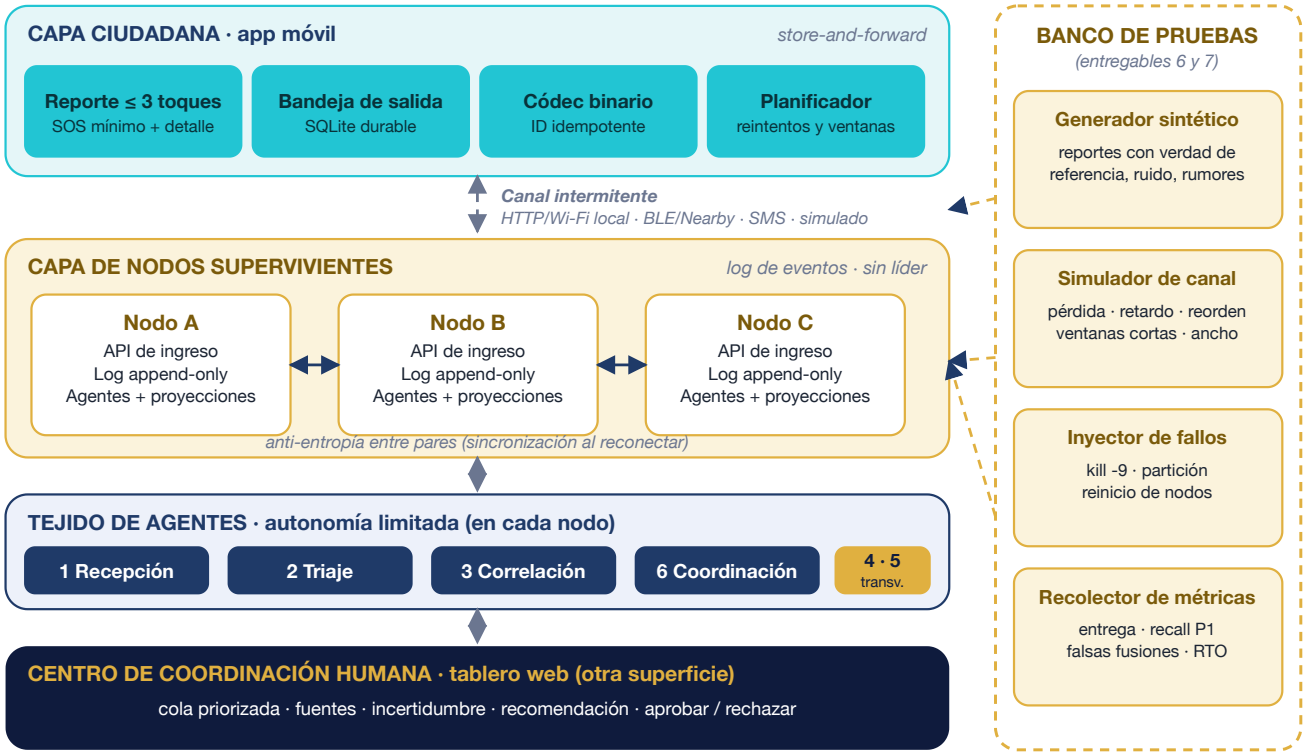


Figura 1. *Componentes del sistema. La app móvil es la capa ciudadana; el tablero es una superficie distinta; el banco de pruebas actúa sobre canales y nodos.*

Tabla 4. *Componentes, responsabilidades y relación con los retos y entregables.*

Componente	Responsabilidad	Ubicación en el repositorio	Reto y entregable
App ciudadana	Capturar, guardar, encolar y transmitir reportes de forma oportunista; mostrar el estado de entrega; ahorrar batería.	`apps/mobile`	Reto 1 · entregable 1
Biblioteca de protocolo	Códec binario, esquemas, identificadores y *vectores de prueba* compartidos por app, nodos y simulador.	`packages/protocol`	Reto 2 · entregable 5
Nodo superviviente	Recibir tramas, guardar el log de eventos, sincronizar con pares, exponer la API del tablero y ejecutar los agentes.	`apps/node`	Retos 3 y 5 · entregable 3
Tejido de agentes	Los seis agentes con sus contratos, validadores y respaldos deterministas.	`packages/agents`	Reto 3 · entregable 4
Tablero del coordinador	Lista priorizada, evidencia, incertidumbre, recomendación y aprobación.	`apps/dashboard`	Reto 6 · entregable 1
Banco de pruebas	Datos sintéticos con verdad de referencia, simulador de canal, inyector de fallos y recolector de métricas.	`packages/sim`	Entregables 6 y 7

3.3 Por qué la app móvil no es todo el sistema

> **DECISIÓN DE ALCANCE**
>
> Has planteado el desarrollo como una app móvil, y eso es correcto para la **capa ciudadana**: es donde el enunciado concentra el Reto 1 y buena parte del Reto 2. Pero los retos 3 a 6 y el recorrido de integración viven fuera del teléfono: los agentes, la sincronización entre nodos y el tablero del coordinador. El coordinador necesita una pantalla amplia y estable (portátil o tableta), no el teléfono de un ciudadano con 15 % de batería.
>
> **Recomendación:** una sola app móvil para ciudadanos, un tablero web para coordinadores y un nodo que ambos pueden alcanzar. Si el equipo es pequeño, el orden de prioridad es: app + transporte, nodo + log, agentes de triaje y correlación, tablero, y por último la sincronización avanzada.

3.4 Pila tecnológica y alternativas

Las decisiones de la columna «Por defecto» son propuestas mías, razonadas para un equipo de hackathon: una sola lengua (Dart) reduce la fricción entre app, nodo, tablero y simulador, y permite compartir el códec y los esquemas como un único paquete de Dart puro (sin Flutter), de modo que los nodos corren con `dart run`. La columna final indica cuándo conviene cambiarlas.

Tabla 5. *Decisiones técnicas por defecto, alternativas y criterio para cambiarlas.*

Decisión	Por defecto	Alternativa	Cuándo cambiarla
App móvil	Flutter (Dart)	React Native; Kotlin nativo	Sólo si se exige un servicio en primer plano que Flutter no cubra: módulo nativo mínimo (ver §5.7).
Almacenamiento local	SQLite vía `package:sqlite3` (WAL, `synchronous=FULL`), con la misma API en app y nodo	Drift; Room (Kotlin)	Si la prueba técnica de §10.1 falla en el móvil.
Nodo	Dart puro + SQLite	Node.js o Python	Si un componente se resuelve mejor en otro lenguaje; el contrato son los esquemas.
Esquemas y protocolo	Paquete Dart puro + vectores de prueba en JSON	Protocol Buffers	Si se quiere generación de código multi-lenguaje; el bit-packing propio es más compacto para el SOS mínimo.
Sincronización	Log append-only + vectores de versiones	CRDT (Automerge, Yjs)	Sólo si se necesitan estados editables concurrentes; aquí los eventos son inmutables.
Tablero	Flutter Web servido por el nodo	Panel nativo de tableta	Si hay requisito de instalación en una tableta sin navegador moderno.
Modelo de lenguaje	API de Claude tras un puerto intercambiable (HTTP directo); entra en B3	Modelo local (p. ej. vía Ollama); sólo reglas	Si no hay Internet ni credenciales en la demo; el puerto ya lo permite.
Pruebas	`dart test` (en VM y en Chrome) + escenarios en YAML con semilla	—	—
Monorepo	*Pub workspaces* de Dart	Melos	Si el espacio de trabajo nativo se queda corto.

3.5 El recorrido de integración de punta a punta

El enunciado exige demostrar al menos este recorrido. La figura lo muestra con la separación de responsabilidades propuesta; la tabla siguiente asocia cada paso con su componente y con la evidencia que deja para el jurado.

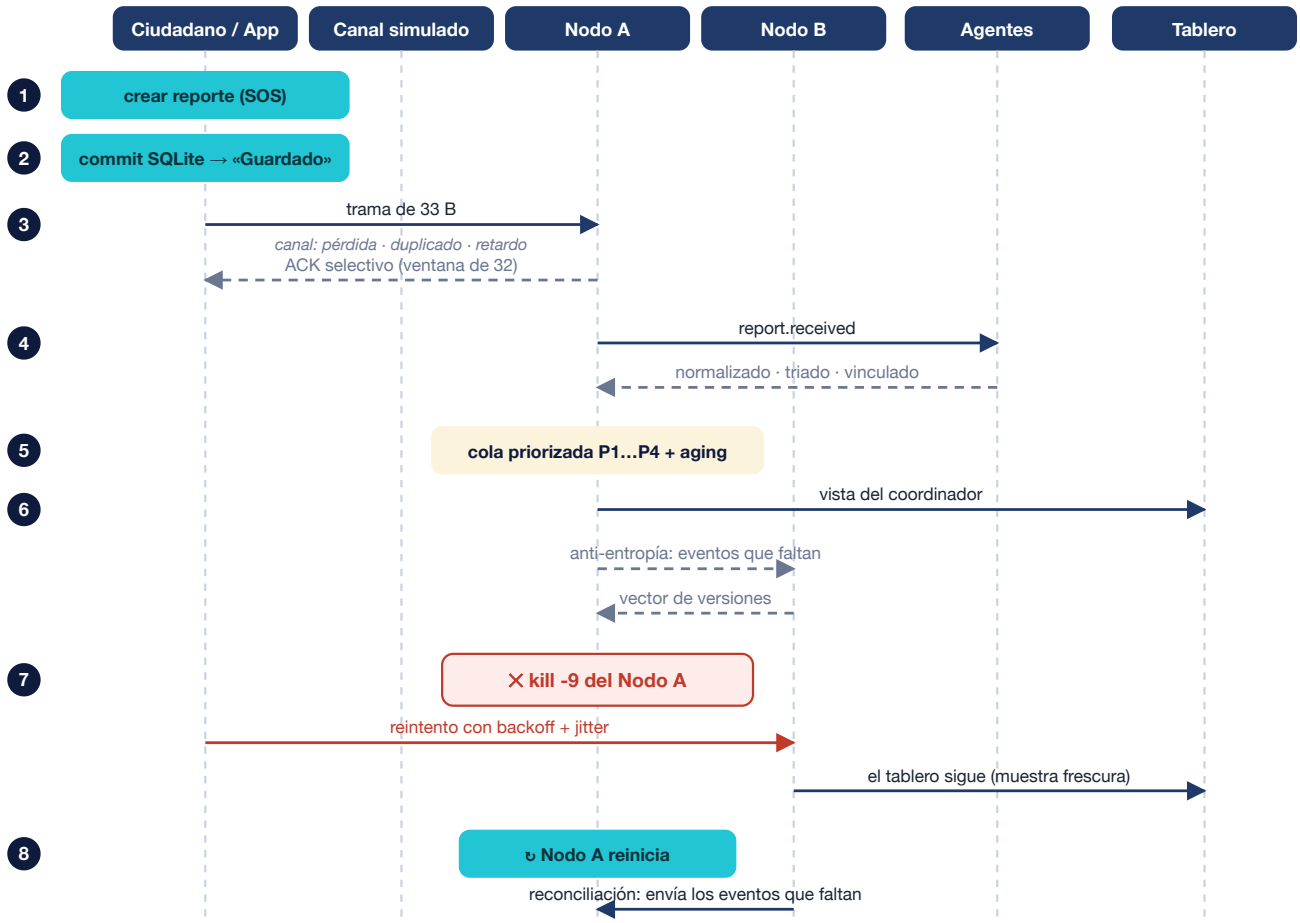


Figura 2. *Recorrido de ocho pasos con la pérdida del Nodo A. Las flechas discontinuas son respuestas; las rojas, reintentos tras la falla.*

Tabla 6. *Cada paso del recorrido mínimo exigido, con su mecanismo, componente y evidencia.*

Paso	Qué ocurre	Mecanismo	Componente	Evidencia generada
1	Reporte ciudadano	SOS en un toque; detalle opcional como revisión del mismo reporte.	App · UI	Captura de pantalla; registro local.
2	Almacenamiento local	Escritura transaccional en SQLite antes de confirmar «Guardado».	App · persistencia	T01: el reporte sobrevive a un cierre brusco.
3	Transmisión intermitente	Trama de 33 B, ACK selectivo, reintentos con espera exponencial y variación.	App · transporte + proxy de canal	T02–T05: curvas de entrega bajo pérdida.
4	Procesamiento por agentes	Agentes 1–3 suscritos al log; respaldo determinista si el modelo falla.	Nodo · agentes	Eventos `report.normalized`, `triage.assessed`, `incident.linked`.
5	Priorización	Cola P1–P4 con envejecimiento; nada se descarta.	Nodo · proyección	T06, T11: recall de P1 y no inanición.
6	Tablero de control	Lista corta + «ver todo», evidencia, incertidumbre, recomendación.	Tablero + agente 6	T14; aprobación registrada.
7	Pérdida del nodo	`kill -9` del Nodo A; el teléfono reintenta contra el Nodo B; el tablero muestra frescura.	Banco de pruebas · nodos	T09: cero reportes con ACK perdidos.
8	Recuperación y sincronización	El Nodo A reinicia, reconstruye sus vistas del log y recibe los eventos que le faltan.	Nodos · sincronización + agente 5	T10: proyecciones idénticas tras la reconciliación.

4 Matriz de trazabilidad

Esta matriz muestra cómo cada requisito del enunciado se vuelve un mecanismo de diseño y una prueba concreta. Es la evidencia de que el análisis cubre el documento completo: si una fila no tuviera prueba, sería un hueco. Los identificadores T01–T16 se definen en el capítulo §9.

4.1 Las diez preguntas de los escenarios

Tabla 7. *Trazabilidad de las diez preguntas de los escenarios.*

ID	Pregunta del enunciado	Respuesta de diseño	Sección	Prueba
E1.1	Crear y guardar un reporte sin conexión	Formulario mínimo y bandeja de salida en SQLite; el «Guardado» se muestra sólo tras el *commit* durable.	§5.4	T01
E1.2	Enviar con el menor número de bytes	Trama binaria de 33 B para el SOS; sin medios; detalles como revisiones pequeñas; fragmentación sólo si hace falta.	§7.2	T02
E1.3	No perderlo si la transmisión se interrumpe	ACK selectivo con ventana, reintentos con espera exponencial y variación, identificador idempotente, estado visible.	§7.4	T03, T04
E1.4	Saber si es una emergencia inmediata	Pista de prioridad local (reglas sobre indicadores marcados) + agente de triaje con evidencia citada y piso de reglas.	§6.4	T06
E1.5	Evitar decenas de registros del mismo incidente	Un reporte vivo por incidente con revisiones; idempotencia en el nodo; correlación semántica; presupuesto por dispositivo que degrada (no rechaza).	§6.6	T05, T07
E2.1	Continuar si el nodo principal se apaga sin aviso	No hay nodo principal: log por nodo, ACK sólo tras escritura durable, reasignación del trabajo por *rendezvous hashing*.	§6.2	T09
E2.2	Sincronizar nodos que trabajaron aislados	Anti-entropía con vectores de versiones; eventos inmutables sin conflictos de escritura.	§6.2	T10
E2.3	Combinar fragmentos sin inventar datos	Los grupos son vistas con vínculos; los campos ausentes se muestran como ausentes; cada dato conserva su origen.	§6.6	T07, T13
E2.4	Identificar y administrar la incertidumbre	Códigos de incertidumbre en el esquema, independencia de fuentes, contradicciones conservadas y mostradas.	§6.6	T08
E2.5	Lista corta sin ocultar casos relevantes	Vista por defecto limitada con contadores visibles de lo que queda fuera; envejecimiento; ningún P1 fuera de la vista inicial.	§6.7	T11, T14

4.2 Los diez objetivos específicos

Tabla 8. *Trazabilidad de los objetivos específicos (numeración del enunciado).*

N.º	Objetivo	Componente y mecanismo	Prueba
1	Captura resiliente	App: bandeja SQLite durable, estados de entrega visibles.	T01
2	Comunicación eficiente	Códec binario, planificador con ventanas, ACK selectivo, fragmentación.	T02–T05
3	Inteligencia agéntica	Seis agentes con fichas, contratos de evento y validadores.	T15
4	Triaje informativo	Agente 2: reglas + modelo, evidencia citada, cola con envejecimiento.	T06, T11
5	Gestión de duplicados	Agente 3: bloqueo, puntuación tri-estado, vista reversible con vínculos.	T07, T08
6	Continuidad operativa	Nodos pares sin líder; agente 5 sin LLM; reanudación desde estado persistido.	T09
7	Coordinación distribuida	Anti-entropía con vectores de versiones al reconectarse.	T10
8	Interacción humana	Tablero con lista corta, evidencia, incertidumbre y aprobación.	T14

N.º	Objetivo	Componente y mecanismo	Prueba
9	Uso responsable de IA	Validadores, citas verificables, sin herramienta de despacho, aprobación humana, auditoría.	T12, T13
10	Validación experimental	Banco de pruebas: generador, simulador de canal, inyector de fallos, métricas.	T01–T16

4.3 Retos, entregables y recorrido de integración

Tabla 9. *Trazabilidad de retos, recorrido de integración y entregables.*

Requisito	Dónde queda resuelto	Evidencia
Reto 1 · Captura ciudadana	App móvil: SOS en un toque, bandeja, modo ahorro (§5).	T01 · pantallas
Reto 2 · Comunicación en ráfagas	Protocolo SARD-WIRE v0 y planificador de envío (§7).	T02–T05
Reto 3 · Tejido de agentes	Seis fichas, flujo por eventos, respaldo determinista (§6).	T06, T12, T13
Reto 4 · La verdad entre fragmentos	Deduplicación, independencia de fuentes, contradicciones (§6.6).	T07, T08
Reto 5 · Sobrevivir a la caída	Log replicado, anti-entropía, reasignación (§6.2).	T09, T10
Reto 6 · El tablero que salva tiempo	Tablero y agente 6 (§6.7).	T14
Escenario 2 · rumores por radio y mensajes a medio terminar	Canal del despachador, `POST /api/dispatch` (§7.8).	T16
Condición de integración (8 pasos)	Recorrido con datos sintéticos y canal simulado (§3.5).	T15 · demostración
Entregables 1 a 7	Tabla de entregables de §2.2.	Repositorio y este documento

5 La app móvil

La app es la capa ciudadana del sistema y el cliente de su transporte. Resuelve el Reto 1 por completo y la mitad emisora del Reto 2. Su criterio de diseño es doble: **nunca perder un reporte** y **gastar lo mínimo** (energía, bytes, atención del usuario).

5.1 Qué hace y qué no hace

La app SÍ hace

- Crear un SOS en un toque y enriquecerlo después.
- Guardar de forma durable antes de confirmar.
- Codificar la trama binaria y mantener la cola de salida.
- Transmitir de forma oportunista y reintentar sin gastar batería.
- Mostrar el estado de entrega en lenguaje claro.
- Sugerir una prioridad de envío con reglas locales (sin LLM).
- Activar un modo de ahorro cuando la batería es baja.

La app NO hace

- Ejecutar modelos de IA ni llamar a un modelo de lenguaje en la ruta de envío.
- Mostrar mapas, fotos, videos ni animaciones.
- Decidir qué es una emergencia: sólo ordena su propia cola.
- Sincronizar con otros teléfonos ni actuar como nodo.
- Exigir registro, contraseña ni cuenta.
- Despachar ni prometer ayuda: informa la entrega, no la respuesta.

5.2 Arquitectura interna

Cinco capas con una regla de dependencia: **la interfaz nunca habla con la red**; sólo escribe en la persistencia, y el transporte lee de la bandeja de salida. Eso hace que la app funcione igual con o sin señal y que el transporte pueda fallar sin afectar la captura. La lógica crítica (códec, cola, planificador) vive en paquetes puros y probados, independientes de la interfaz.

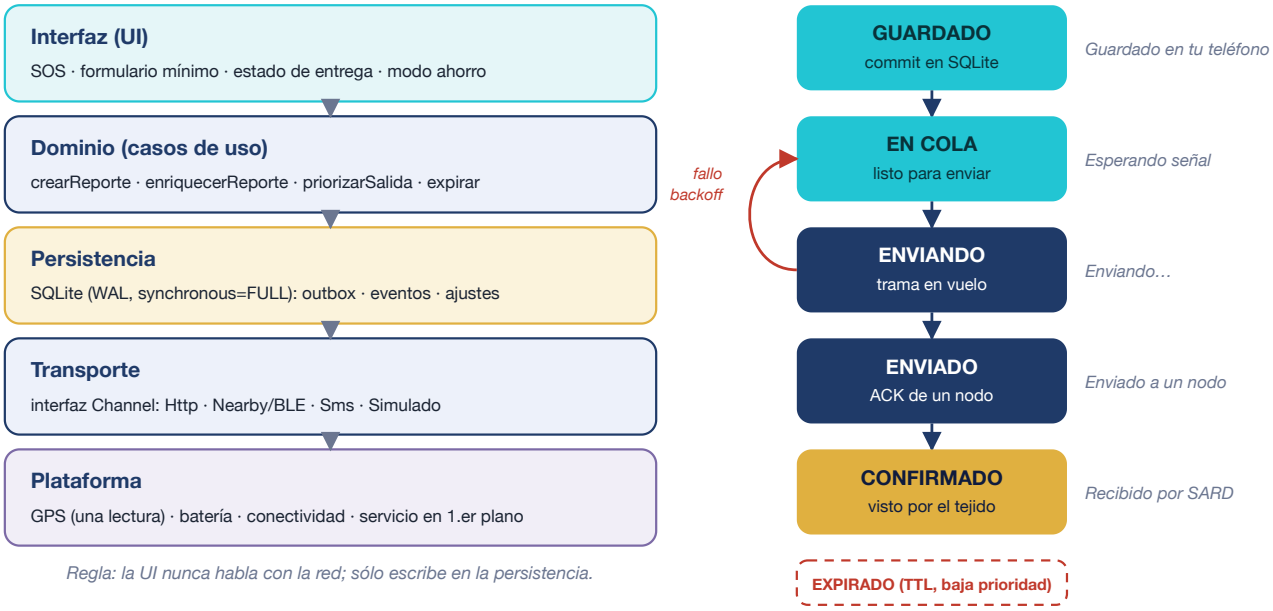


Figura 3. *Capas de la app (izquierda) y máquina de estados de entrega con el texto que ve el ciudadano (derecha).*

5.3 Experiencia: reportar en tres toques

La pregunta de innovación del Reto 1 es cómo ayudar a una persona asustada a informar lo esencial con la menor interacción y el menor consumo. La respuesta propuesta es el **reporte progresivo**: lo esencial se envía primero y todo lo demás es opcional.

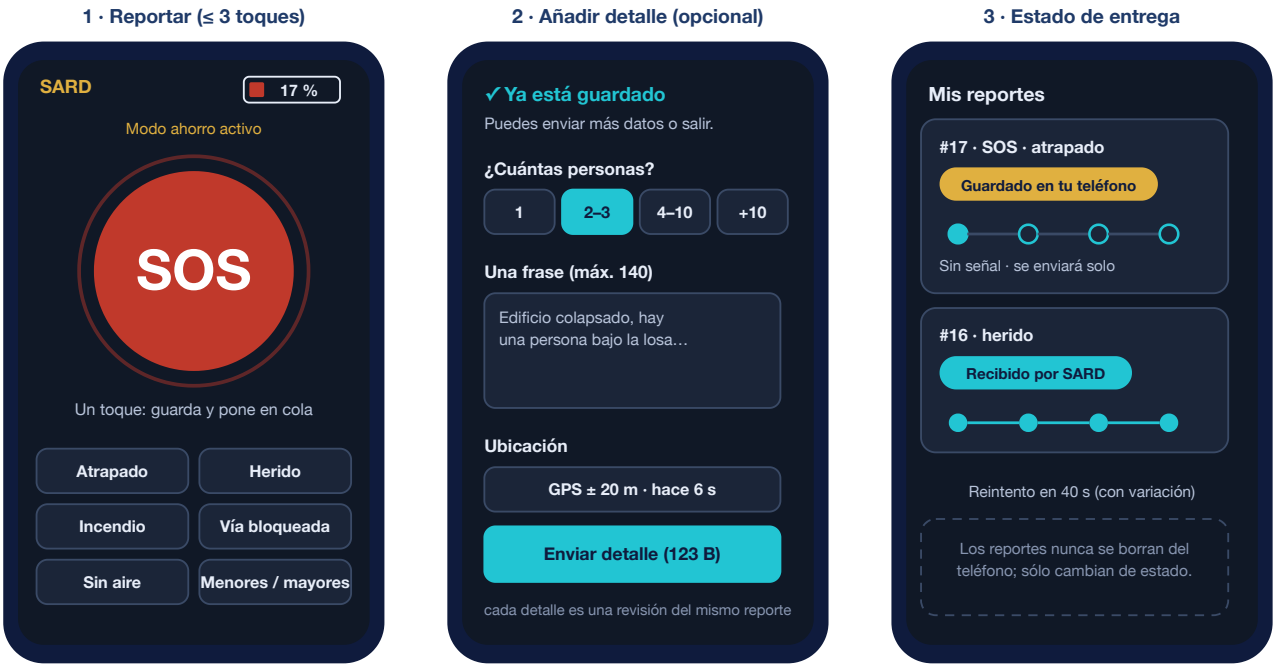


Figura 4. *Wireframes de la app. Tema oscuro (menos energía en pantallas OLED), texto grande, sin imágenes ni animaciones.*

1. **Un toque** en *SOS*, o en uno de los seis indicadores de situación, crea la revisión 0: tipo, ubicación (una sola lectura), hora, nivel de batería e indicadores. Se guarda y entra a la cola *en ese instante*.
2. **Un segundo toque** (opcional) elige el rango de personas: si se marca antes del SOS viaja en la revisión 0; si es después, sale enseguida como una revisión del mismo reporte, de pocos bytes.
3. **Una frase** (opcional, hasta 140 bytes) sólo se ofrece cuando ya hay un aviso vivo, y sale como otra revisión: el SOS *nunca* lleva texto. Cada revisión reutiliza el mismo identificador de reporte: *enriquece*, no duplica.

DECISIONES DE EXPERIENCIA

- **Ubicación honesta.** Una sola lectura con tiempo máximo. Si falla, se usa la última conocida marcada con su edad, o una referencia manual en texto. La app *nunca* inventa coordenadas; el campo viaja con su calidad (±10 m, ±50 m, …, manual, ausente).
- **Permisos antes de la emergencia.** El permiso de ubicación se pide en el primer arranque, no durante un sismo.
- **Un campo nunca se quita.** Cada revisión sólo añade o corrige campos (último valor gana por campo); el historial se conserva en el teléfono.
- **Accesibilidad.** Botones de gran tamaño, contraste alto, lectura por lector de pantalla y mensajes cortos en lenguaje llano.

Interfaz implementada (B2). Los wireframes de arriba son el boceto de partida; lo que se construyó y se probó a mano en el emulador cambió en estos puntos, todos pensados para una persona con miedo, prisa y poca batería:

- **Un aviso, una tarjeta.** La bandeja guarda una fila por trama, pero la persona ve *un* aviso: el SOS y todo lo que se le añade (la ubicación que llega después, los indicadores) se pliegan en una sola tarjeta. Desaparecen las filas «Detalle #N» que nadie creó.

- **El estado, justo debajo del botón.** Tras tocar SOS, una tarjeta de color (ámbar «guardado», verde «enviado»; siempre con icono y texto) dice qué pasa y qué hacer —casi siempre *nada*—: «Sale solo en cuanto haya señal. No hace falta volver a tocar.» Los textos de estado son los de la tabla de abajo, sin cambios.
- **Primero lo urgente, después el detalle.** Los indicadores pueden marcarse antes del SOS o hasta 10 minutos después; los que se marcan después salen enseguida como una actualización del mismo aviso. Lo ya enviado se ve marcado en verde y no se puede quitar.
- **Franja de «sin señal»** que dice con claridad que el SOS se guarda y saldrá solo.
- **Nada se mueve bajo el dedo.** Cuadrícula de dos columnas con indicadores de tamaño fijo y una zona de estado de altura constante: ni el GPS que llega, ni el acuse del nodo, ni un detalle añadido desplazan los botones (la primera versión sí lo hacía y se descubrió usándola; ahora lo protegen pruebas).
- **Un icono por situación**, texto de al menos 14–15 sp, objetivos táctiles de 64 dp, vibración al tocar SOS y etiquetas para el lector de pantalla («Atrapado, ya enviado»). El permiso de ubicación se pide *después* de pintar la primera pantalla.

Personas y frase, implementadas (B4). Lo que se construyó y se verificó (a mano en el emulador y con un recorrido automático en Android e iOS contra un nodo real) difiere del boceto en lo siguiente:

- **La escala de personas la fija esta versión** (el diseño sólo reservaba 3 bits): 0 no indicado, 1, 2–5, 6–20, 21–100 y más de 100; 6 y 7 quedan reservados y un nodo los lee como «no indicado». «No indicado» es lo que lleva un SOS sin detalle y no es una opción que se toque. Se puede *corregir* el rango tras el SOS (manda el último que indica algo, también si es menor), pero no «des-marcarlo» una vez enviado.
- **El campo de la frase corta por bytes, no por letras.** Una «ñ» pesa 2 bytes y un emoji 4 o más: el filtro de entrada impide pasar de 140 B UTF-8 y recorta por *grafemas* completos (un emoji o una letra con su acento combinante entra entero o no entra). El texto escrito sobrevive al refresco de cada segundo (cada cinco en ahorro), y personas y frase *desaparecen* en «sólo SOS».
- **Nada se mueve bajo el dedo, tampoco aquí.** Los rangos y el cuadro de la frase tienen tamaño fijo, marcados o no, con o sin texto; una prueba mide posiciones antes y después de cada cambio.
- **Costes asumidos.** El historial «Tus avisos» queda ≈ 330 dp más abajo; con el teclado abierto no se fija el SOS arriba (ocuparía ~390 dp de los ~640 que quedan): arrastrar la lista cierra el teclado y lo deja a un gesto de distancia. Verificado con el teclado flotante del emulador, no con un teclado físico ni con lector de pantalla.

5.4 Datos locales y estados de entrega

La bandeja de salida es una tabla SQLite. La regla de oro: **la pantalla sólo muestra «Guardado» después de que la transacción confirme** con `synchronous=FULL`, de modo que un cierre brusco de la app no puede perder lo que el usuario ya vio como guardado. (Las pruebas del proyecto verifican la muerte del proceso; `synchronous=FULL` es la garantía de SQLite frente a un corte de energía, que no se somete a prueba aquí.)

```
-- Esquema local (SQLite). PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL;
CREATE TABLE report (
  report_id  BLOB    PRIMARY KEY,   -- 12 B: device_id(8) || seq de la trama rev 0 (4)
  rev        INTEGER NOT NULL,      -- última revisión local
  fields     TEXT    NOT NULL,      -- JSON: valor vigente de cada campo
  created_at INTEGER NOT NULL
);
CREATE TABLE outbox (
  report_id   BLOB    NOT NULL,
  rev         INTEGER NOT NULL,
  frame_seq   INTEGER NOT NULL UNIQUE, -- contador de tramas: lo que acusa el ACK
  frame       BLOB    NOT NULL,      -- trama ya codificada
  prio_hint   INTEGER NOT NULL,      -- 0..3, sólo orden de envío
  state       TEXT    NOT NULL,      -- QUEUED | SENDING | SENT | CONFIRMED | EXPIRED
  attempts    INTEGER NOT NULL DEFAULT 0,
  next_try_at INTEGER,               -- null = ahora
  ttl_at      INTEGER,
  last_error  TEXT,
  PRIMARY KEY (report_id, rev)
);
CREATE INDEX outbox_pending ON outbox(state, next_try_at);
```

Tabla 10. *Estados de la bandeja de salida y el texto mostrado al usuario.*

Estado interno	Texto para el ciudadano	Significado y transición
QUEUED (intentos = 0)	«Guardado en tu teléfono»	Confirmado en disco; aún no se ha intentado enviar.
QUEUED (intentos > 0)	«Esperando señal»	Hubo un intento fallido; hay un reintento programado con espera aleatoria.
SENDING	«Enviando…»	La trama está en vuelo; si vence el temporizador sin ACK vuelve a QUEUED.
SENT	«Enviado a un nodo»	Un nodo confirmó haberla guardado de forma durable (ACK).
CONFIRMED	«Recibido por SARD»	El nodo confirmó con un `STATUS` que el reporte está en la cola priorizada **y**, si hay al menos dos nodos vivos, que existe en dos de ellos (ver §7.3).
EXPIRED	«Se enviará cuando haya señal»	Venció el TTL del reintento activo: pasa a envío oportunista de baja frecuencia. **Nunca se borra del teléfono.**

5.5 Planificador de envío

El enunciado pregunta cómo lograr que la información crítica llegue aunque la ventana de comunicación sea muy breve. El planificador se basa en tres ideas: **aprovechar cada ventana**, **enviar lo más urgente primero sin dejar morir lo demás** y **no derrochar batería esperando**.

- **Disparadores.** Cambio de conectividad (la señal reaparece), apertura de la app, un reintento programado y la creación de un reporte nuevo. No hay sondeo periódico agresivo.
- **Ventanas.** El planificador aprende cuánto suelen durar las ventanas (promedio móvil exponencial) y limita el tamaño del lote a lo que cabe en ellas. Ante una ventana abierta, **envía en ráfaga sin esperar un ACK por trama** (ventana deslizante de hasta 8 tramas).
- **Orden.** Prioridad efectiva = prioridad de la pista local *menos* envejecimiento por edad. Además, una cuota fija reserva una de cada cinco posiciones del lote para el pendiente más antiguo: la urgencia no deja morir los casos leves.

- **Reintentos.** Espera exponencial con *variación completa*: `espera = aleatorio(0, mín(tope, base × 2^intentos))`, con base de 2 s y tope de 300 s como valores iniciales a calibrar. La variación evita que miles de teléfonos reintenten a la vez cuando vuelve la señal.
- **Segundo plano.** Las tareas periódicas del sistema operativo son demasiado gruesas para ventanas breves (en Android, el mínimo de una tarea periódica es de 15 minutos). Por eso el planificador se apoya en eventos de conectividad mientras el proceso vive y, durante una emergencia declarada por el usuario, en un servicio en primer plano.

```
// Pseudocódigo del planificador (valores iniciales, a calibrar con T03/T04)
al ocurrir(conectividad | reintento | reporte_nuevo | app_abierta):
    ventana = estimarVentana()                    // promedio móvil de duraciones previas
    lote    = seleccionar(outbox, presupuesto = ventana.bytes)
    para cada trama en lote (hasta 8 en vuelo):
        enviar(trama); estado = SENDING
al recibir ACK(ack_seq, bitmap): marcar SENT las tramas cubiertas
al vencer temporizador(trama): intentos++; próximo = ahora + aleatorio(0, mín(300 s, 2 s × 2^intentos))

seleccionar: ordenar por prioridad_efectiva = prio_hint − envejecimiento(edad)
             y reservar 1 de cada 5 posiciones para el pendiente más antiguo
```

5.6 Ahorro de batería

El enunciado fija el umbral del problema: teléfonos «con menos del 20 % de batería». Se proponen dos modos automáticos, además del uso responsable por diseño (sin medios, sin animaciones, una sola lectura de GPS).

Tabla 11. *Modos de energía de la app. El umbral del 5 % es una propuesta a validar con pruebas en un teléfono real.*

Modo	Se activa	Comportamiento
Normal	Batería ≥ 20 %	Estados y mensajes completos; reintentos con los parámetros base.
Ahorro	Batería < 20 % (umbral del enunciado)	Tema oscuro fijo, sin animaciones, sin sondeo de red, lotes más grandes y menos frecuentes, lectura de ubicación única, brillo reducido si el sistema lo permite.
Sólo SOS	Batería < 5 % (propuesta mía)	Una sola pantalla con el botón SOS; el envío inmediato y oportunista es lo único que sigue activo.

5.7 Elección del framework móvil

Los requisitos duros de la app son tres: **cola persistente transaccional**, **reintento fiable en segundo plano** y **codificación binaria compacta**. Ningún framework los cumple «de fábrica»; las tres opciones habituales los alcanzan con distinto esfuerzo.

Tabla 12. *Comparación cualitativa. No es una medición: es el análisis de compromisos que respalda la decisión.*

Criterio	Flutter (Dart) · elegido	React Native + Expo	Kotlin nativo (Android)
Segundo plano en Android	Medio: plugin de WorkManager; servicio en primer plano con un módulo nativo mínimo.	Medio: módulos de terceros; módulo nativo mínimo.	Alto: WorkManager y servicios en primer plano directos.
SQLite transaccional	Bueno: `package:sqlite3` (misma API que el nodo) o Drift.	Bueno (expo-sqlite u otras).	Excelente (Room).
Binario y enteros de 64 bits	Bueno en móvil y VM (64 bits). **En Web son de 53 bits**: `device_id` como 8 bytes o hexadecimal, nunca como `int`.	Aceptable: `Uint8Array`; 64 bits con `BigInt`.	Excelente: `Long`.

Criterio	Flutter (Dart) · elegido	React Native + Expo	Kotlin nativo (Android)
Compartir código con nodo y tablero	**Alto**: Dart en app, nodo y Flutter Web con un paquete puro.	Alto si el nodo es Node.js.	Bajo.
Consumo de energía	Bueno.	Aceptable con motor Hermes y sin animaciones.	El mejor.
Alcance en iOS	Sí (con las limitaciones del sistema).	Sí.	No (otra app).
Curva para este proyecto	**Nula:** ya se domina.	Media.	Media-alta.

DECISIÓN Y CÓMO PROTEGERLA

Flutter (Dart) de punta a punta. El proyecto lo desarrolla una sola persona que ya domina Flutter; con ese dato, el criterio dominante es la velocidad de un único desarrollador y un solo lenguaje para app, nodos, tablero y simulador. Se protege con dos reglas: (1) los paquetes compartidos (`protocol` , `agents` , `sim`) son **Dart puro**, sin importar Flutter, de modo que los nodos corren con `dart run` y las pruebas no necesitan dispositivo; (2) el códec se especifica con *vectores de prueba* en JSON (entrada → bytes esperados) que se ejecutan tanto en la VM como en Chrome. Si el servicio en primer plano resulta imprescindible en Android, se añade un módulo nativo muy pequeño.

5.8 Riesgos específicos del móvil

Tabla 13. *Riesgos de la app móvil y su mitigación.*

Riesgo	Mitigación
iOS limita el trabajo en segundo plano	Demostración en Android; en iOS, enviar en primer plano y avisar al usuario de que mantenga la app abierta.
La optimización de batería del fabricante mata servicios	Pedir la exclusión durante la instalación; probar en el dispositivo real, no sólo en emulador.
Permiso de ubicación denegado o tardío	Pedirlo en el primer arranque; flujo alternativo con referencia manual; calidad de ubicación en la trama.
Reloj del teléfono incorrecto	La trama lleva `capturado_en` ; el nodo registra su hora de llegada y el desfase estimado; el orden interno no depende del reloj del dispositivo.
Enteros de 64 bits en Dart compilado a Web	Los `int` son de 64 bits en la VM pero de 53 en JavaScript, y el tablero (Flutter Web) comparte el protocolo. Modelar `device_id` como 8 bytes o hexadecimal, mantener cada campo empaquetado en 32 bits o menos y ejecutar los vectores con `dart test -p vm,chrome` .
Abuso o reportes falsos	Presupuesto por dispositivo que degrada (no rechaza), etiqueta de integridad y peso de corroboración por independencia de fuentes (§6.6).
Privacidad	Sin cuenta obligatoria; ubicación aproximada cuando se envía a servicios externos; aviso claro de qué se envía.

6 Nodos supervivientes y tejido de agentes

Este capítulo cubre los retos 3 a 6: dónde viven los datos, cómo se sincronizan, cómo colaboran los agentes y cómo se presenta el resultado al coordinador. Respalda los entregables 3 (arquitectura) y 4 (fichas de agentes).

6.1 El nodo superviviente

Un nodo es un proceso pequeño que puede funcionar **completamente solo**: sin pares, sin Internet y sin modelo de lenguaje. Su corazón es un **log de eventos que sólo crece** (*append-only*, en SQLite). Todo lo demás —la cola priorizada, los grupos de incidentes, el estado de los nodos— son *proyecciones* que se pueden reconstruir desde cero leyendo el log.

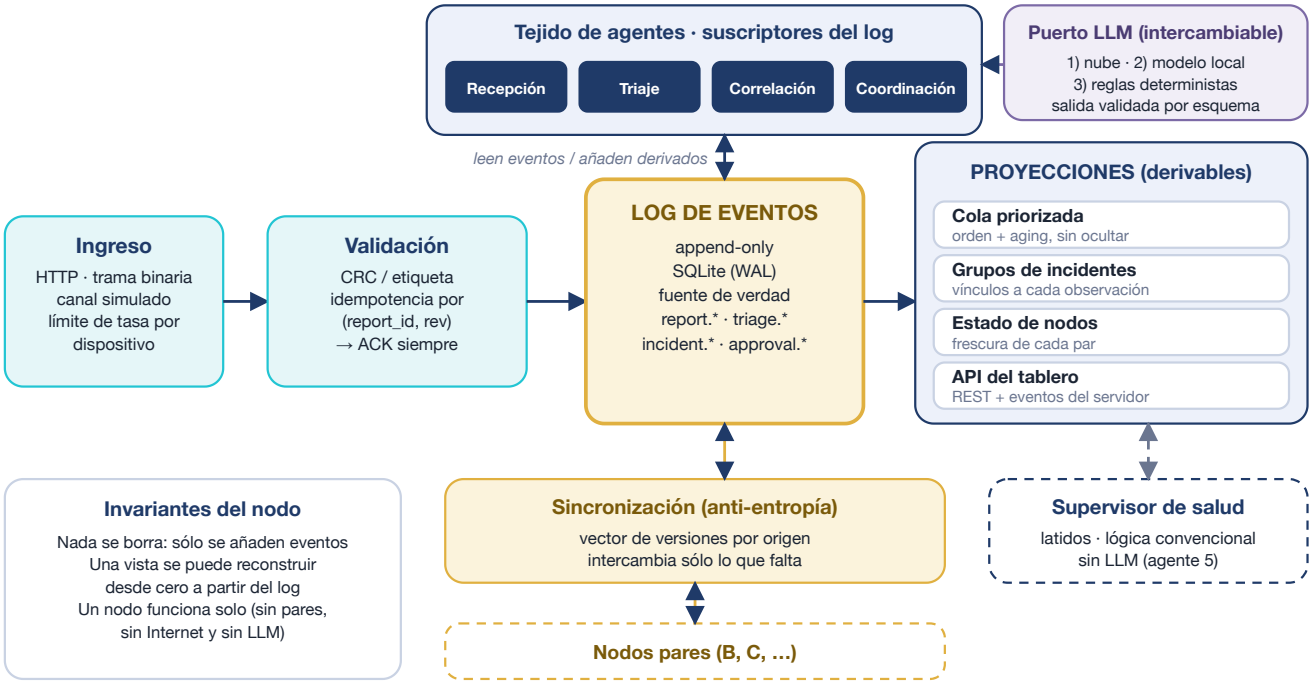


Figura 5. *Arquitectura interna de un nodo. Los agentes son suscriptores del log: leen eventos y añaden eventos derivados.*

Esta decisión (*event sourcing* con un log local por nodo) responde a casi todo el Escenario 2 a la vez: la persistencia es el log; la recuperación es releerlo; la sincronización es intercambiar los eventos que faltan; y la trazabilidad es gratuita porque cada salida enlaza los eventos de los que proviene.

Modelo de evento

```
{
  "event_id": "B:000412",                      // nodo de origen : contador propio
  "type": "triage.assessed",
  "subject": { "report_id": "9a3c51e07b2d44f1-17", "rev": 1 },
  "hlc": "2026-08-10T14:09:02.114Z-0003-B",    // reloj lógico híbrido: orden causal sin fiarse del
reloj
  "producer": { "agent": "triage", "version": "0.3.1", "mode": "rules+llm", "model": "claude-sonnet-5-
5" },
  "inputs": ["A:000377", "B:000411"],                // eventos de los que depende (trazabilidad)
  "body": {
    "urgency": "P1", "confidence": 0.82,
    "signals": [ { "code": "atrapado",  "evidence": { "field": "flags.atrapado" } },
                 { "code": "texto_losa", "evidence": { "quote": "atrapada bajo la losa" } } ],
    "uncertainty": ["fuente_unica"], "escalate_to_human": false
  }
}
```

Tabla 14. *Catálogo de eventos. Cada uno referencia los eventos de los que depende.*

Tipo de evento	Productor	Contenido clave
`report.received`	Nodo (ingreso)	Trama decodificada, canal, hora de llegada y desfase de reloj estimado. Se escribe *antes* del ACK.
`report.normalized`	Agente 1	Campos validados, faltantes y desconocidos; ubicación con su origen y calidad.
`triage.assessed`	Agente 2	Urgencia, confianza, señales con cita, códigos de incertidumbre, escalamiento.
`incident.linked` / `.unlinked`	Agente 3 o humano	Grupo, miembros, estado (*same* / *possible* / *different*), contradicciones.
`recommendation.proposed`	Agente 6	Acción, recursos existentes, justificación citada, requisitos pendientes.
`approval.granted` / `.denied`	Humano (tablero)	Quién, cuándo, a qué recomendación y con qué nota.
`node.health`	Agente 5	Estado de pares, retraso de sincronización, tareas reasignadas.
`delivery.status`	Agente 4	Confirmación de segundo nivel hacia el teléfono (`STATUS` : en cola priorizada y número de réplicas).

6.2 Sincronización, pérdida de nodo y recuperación

La arquitectura es **sin líder y eventualmente consistente**: prioriza seguir funcionando y converger después sobre mostrar una vista única en todo momento. Es la elección correcta para este escenario porque los nodos *van a* quedar aislados; lo importante es que no se pierda nada y que se vea con honestidad qué tan fresca es cada vista.

Anti-entropía con vectores de versiones

- Cada nodo numera sus propios eventos (`B:000412`). Su *vector de versiones* dice, por cada origen, hasta qué contador tiene todo.
- Al contactar a un par: `HAVE(vector)` → el par responde con `EVENTS` sólo de los rangos que faltan. Cuesta lo que falta, no el total.
- Los eventos son inmutables y tienen identificador único: aplicarlos dos veces o en otro orden produce el mismo estado.

Pérdida de un nodo sin aviso

- Un reporte sólo recibe ACK **después** de escribir `report.received` con escritura durable. Un nodo que cae no pierde lo que ya acusó.
- El teléfono, sin ACK, reintenta contra otro nodo; la idempotencia absorbe el duplicado.
- El agente 5 detecta la ausencia por latidos y reasigna el trabajo al siguiente nodo en el orden de *rendezvous hashing*.

Quién procesa cada reporte

Para no calcular lo mismo en todos los nodos, cada reporte tiene un **dueño**: el nodo vivo con mayor `hash(nodo, report_id)` (*rendezvous hashing*). Si el dueño no produce el evento esperado dentro de un plazo, el siguiente en el orden lo toma. Si por una falsa alarma dos nodos lo procesan, no pasa nada grave: los eventos de salida son idempotentes y sólo se desperdicia cómputo.

Implementado (B5). El peso es el SHA-256 de `nodo|reporte` (primeros 8 bytes), fijado con valores de oro calculados aparte; no se usa `String.hashCode`. El dueño gobierna *sólo el trabajo caro y repetible* —la consulta al modelo y la auditoría de vínculos—: recibir, guardar, acusar y calcular las reglas siguen siendo de cada nodo, así que perder al dueño nunca deja a nadie sin respuesta. El dueño procesa también lo que le llega *por replicación* (descubierto al revisar el diseño: sin eso el «relevo» era el receptor haciéndolo todo tras esperar); el siguiente espera `posición × plazo` (10 s por omisión) y no consulta si el juicio del dueño llegó replicado antes. Medido: con dos modelos sanos, de 40 reportes que entran por A, A hace 24 y B 16 sin repetir ninguno; con el modelo de B colgado, A los cubre todos; tras `kill -9` de B, A es dueño de todo al instante (`docs/resultados/t12.md`).

Conflictos entre agentes

Dos nodos aislados pueden emitir juicios distintos sobre el mismo reporte (el modelo de lenguaje no es determinista). Ambos eventos se conservan. La proyección aplica una regla **conservadora**: para la urgencia muestra el valor más alto y marca «juicios discrepantes»; para el triaje de personas es preferible un falso positivo revisable que un falso negativo.

Recuperación de un nodo

1. El proceso reinicia y reconstruye sus proyecciones releyendo el log local.
2. Envía su vector de versiones a los pares vivos y recibe los eventos que le faltan.
3. El agente 5 compara y publica `node.health`; el tablero deja de mostrar «vista parcial».

> **LO QUE ESTA ARQUITECTURA NO PROMETE (Y DEBE DECIRSE EN LA DEMOSTRACIÓN)**
> Un reporte con ACK está a salvo mientras el disco de ese nodo sobreviva; si el disco se destruye antes de la primera sincronización, el evento se pierde. Se reduce la ventana con la confirmación de segundo nivel: el teléfono sólo marca «Recibido por SARD» cuando el `STATUS` indica que el reporte está en la cola priorizada y, si hay dos o más nodos vivos, replicado en al menos dos. Además, durante un aislamiento las vistas de dos tableros pueden diferir; por eso cada vista muestra su **frescura**.

6.3 El tejido de agentes

El enunciado sugiere seis agentes: un flujo principal (recepción → triaje → correlación → coordinación) apoyado por dos servicios transversales (comunicación y resiliencia). Se implementan como suscriptores del log con contratos de evento tipados. **No hay agente central**: cada nodo ejecuta el tejido completo sobre su propio log.

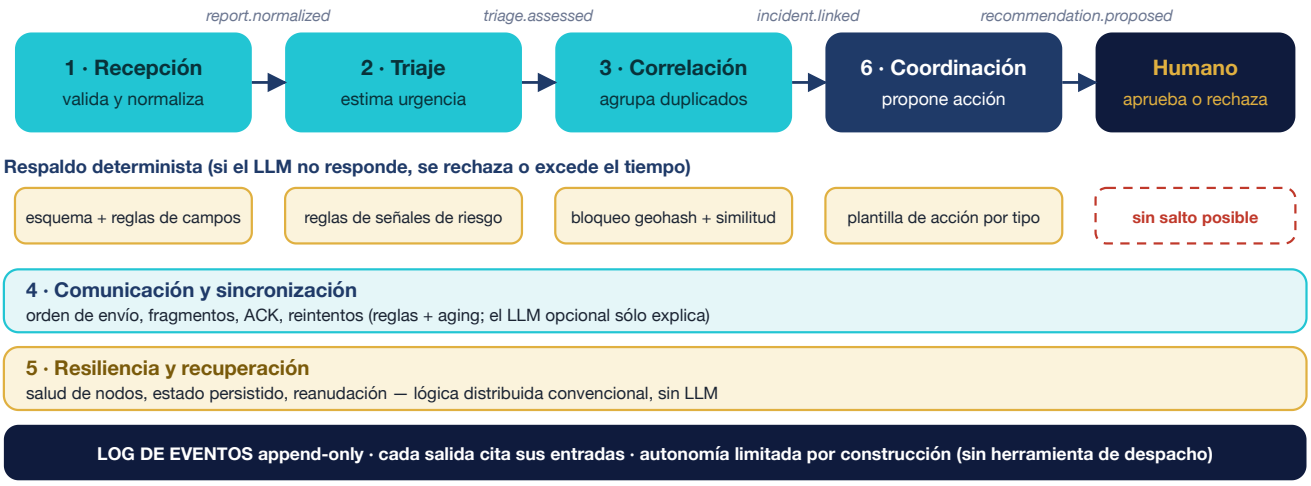


Figura 6. *Flujo de eventos entre agentes, respaldo determinista de cada uno y servicios transversales.*

Reglas comunes a todos los agentes:

- **Entradas y salidas tipadas.** Cada salida se valida contra un esquema antes de entrar al log; lo que no valida se rechaza y queda registrado.
- **Idempotencia.** Un agente produce a lo sumo un evento por (sujeto, versión del agente). Reprocesar no duplica.
- **Plazos y reintentos acotados.** Toda llamada a un modelo tiene tiempo máximo; tras el límite se usa el respaldo y el evento lo indica (`mode: "rules"`).
- **Sin efectos laterales.** Ningún agente puede despachar, notificar a terceros ni modificar el log más allá de sus propios eventos.

6.4 Fichas de los agentes

Una ficha por agente con los ocho campos que pide el entregable 4. El «modelo sugerido» se justifica en §6.5.

Agente 1 · Recepción y normalización

Objetivo	Convertir cada observación en un reporte normalizado y validado, con los campos faltantes explícitos.
Entradas	`report.received`: carga decodificada, canal, hora de llegada, desfase de reloj estimado. Dos canales de entrada: la trama de la app y el informe de un despachador (§7.8), que llega sin ubicación y como fuente de segunda mano.
Salidas	`report.normalized`: campos, errores de validación, campos desconocidos, campos faltantes, ubicación (valor o nulo, origen, calidad).
Herramientas	Validador de esquema; decodificador de trama; extractor de texto libre (modelo, opcional).
Memoria	Ninguna entre reportes. Lee catálogos de tipos y de lugares conocidos (sólo lectura).
Límites de autonomía	No inventa ni completa la ubicación ni cifras. Lo extraído del texto se marca `inferido` y no sobrescribe campos del teléfono.
Manejo de errores	Trama corrupta o truncada → `estado: incompleto` con la razón; sigue en la cola. Si el modelo falla, sólo se usan los campos estructurados.
Terminación	Un `report.normalized` por (reporte, revisión). Tras dos fallos consecutivos pasa a la bandeja de errores y avisa al agente 5.
Respaldo · modelo	Totalmente determinista. Modelo sugerido (sólo texto libre): `claude-haiku-5-5`.

Agente 2 · Triaje y priorización

Objetivo	Proponer un nivel de urgencia (P1–P4) con evidencia, confianza y motivo de escalamiento.
Entradas	`report.normalized` y el historial del mismo reporte.
Salidas	`triage.assessed`: urgencia, confianza (0–1), señales con cita, códigos de incertidumbre, `escalate_to_human`, versión.
Herramientas	Reglas de señales de riesgo (configuración versionada); modelo con salida estructurada; verificador de citas (cada cita debe ser subcadena del texto o un campo existente).
Memoria	Ninguna. La tabla de señales es configuración.
Límites de autonomía	Herramienta de apoyo, no diagnóstico clínico. **Piso por reglas:** «atrapado», dificultad respiratoria e incendio imponen al menos P2 aunque el modelo discrepe. Ante la duda sube a revisión, nunca baja la urgencia por falta de datos. **Implementado (B3):** un SOS sin banderas ni texto —o con un texto que las reglas no entienden— sale en **P3** con `sin_detalle` o `texto_no_interpretado` y `escalate_to_human`; sólo un texto que niega expresamente el peligro («sin heridos») sin ninguna señal positiva baja a P4.
Manejo de errores	Salida fuera de esquema o cita inválida → se descarta la parte del modelo. Rechazo (`refusal`) o tiempo agotado → sólo reglas, con `mode: "rules"`.
Terminación	Un evento por (reporte, revisión, versión). Se reevalúa cuando llega una revisión mayor o una corroboración nueva.
Respaldo · modelo	Reglas deterministas. Modelo sugerido: `claude-sonnet-5-5`.

Agente 3 · Correlación y deduplicación

Objetivo	Agrupar observaciones que probablemente describen el mismo suceso, medir la independencia de las fuentes y preservar las contradicciones.
Entradas	`report.normalized`, `triage.assessed` e índice espacial-temporal local de incidentes abiertos.
Salidas	`incident.linked` / `.unlinked`: grupo, miembros, similitud, estado (same / possible / different), contradicciones, corroboración.
Herramientas	Bloqueo por *geohash* vecino, ventana temporal y tipo; puntuación determinista; detector de copias de texto; modelo sólo para adjudicar pares «possible» y extraer afirmaciones.
Memoria	Índice de incidentes abiertos (derivable del log).
Límites de autonomía	Nunca borra ni modifica observaciones. «same» sólo sobre un umbral alto; «possible» se muestra como sugerencia. Toda fusión es reversible y conserva los vínculos.
Manejo de errores	Sin ubicación: agrupa sólo con umbral más estricto y marca `ubicacion_ausente`. Sin modelo: decide el puntaje y las contradicciones figuran como «no evaluadas».
Terminación	Evalúa como máximo K candidatos por reporte nuevo y termina; se reabre si llegan observaciones nuevas del grupo.
Respaldo · modelo	Puntaje determinista. Modelo sugerido: `claude-sonnet-5-5`.

Agente 4 · Comunicación y sincronización (servicio transversal)

Objetivo	Decidir qué se transmite primero y administrar colas, fragmentos, reintentos y confirmaciones, tanto en el teléfono como en el nodo.
Entradas	Estado de las colas, estimación de ventana y ancho de banda, política operativa, urgencia (pista local o triaje).
Salidas	Plan de transmisión (lote ordenado), estado de entrega, eventos pendientes de sincronización, `delivery.status`.
Herramientas	Planificador con envejecimiento; fragmentador y ensamblador; temporizadores de retransmisión. No requiere modelo.
Memoria	Estado de las colas, persistido.
Límites de autonomía	La priorización no hace desaparecer los casos de baja prioridad: envejecimiento y cuota mínima. Nada se descarta sin dejar un evento.
Manejo de errores	Canal caído → reprograma con espera exponencial y variación. Trama inválida → NACK y contador por dispositivo.
Terminación	Un mensaje termina con su ACK, su vencimiento o un descarte registrado. El planificador es un servicio continuo.
Respaldo · modelo	Lógica determinista. El modelo, si se usa, sólo redacta la explicación para el tablero.

Agente 5 · Resiliencia y recuperación (servicio transversal, sin LLM)

Objetivo	Supervisar la salud de los nodos, detectar interrupciones y reanudar el procesamiento desde el estado persistido; reportar conflictos de sincronización.
Entradas	Latidos de pares, errores de procesos, huecos en los vectores de versiones.
Salidas	`node.health` (estado, última vista, retraso), tareas reasignadas, conflictos detectados.
Herramientas	Detector de fallos por latidos con umbral; *rendezvous hashing* para reasignar; reanudación desde cursores persistidos.
Memoria	Cursores por suscriptor y vector de versiones, persistidos.
Límites de autonomía	**Sin modelo de lenguaje:** lógica distribuida convencional, como indica el enunciado. Decide sobre infraestructura propia, nunca sobre recursos de respuesta.
Manejo de errores	Una falsa alarma de caída se absorbe por idempotencia (sólo cuesta cómputo repetido).
Terminación	Servicio continuo; cada tarea reasignada termina cuando existe el evento resultado.
Respaldo · modelo	No aplica: no usa modelo.

Implementado (B5). La parte observable: el estado de los pares vive *en memoria* (`/api/nodes`: sincronizado, atrasado, sin respuesta tras tres periodos, desconocido) y **no** se escribe un evento por latido —cada uno invalidaría la caché de la cola y haría crecer el log sin límite—. Tras reiniciar, todo par es «desconocido» y la vista se declara parcial hasta hablar con él; un nodo sin pares no declara vista parcial. Las tareas reasignadas se ven en el reparto del agente de arriba; no hay detector de fallos por latidos distinto del sondeo de replicación.

Agente 6 · Coordinación de respuesta

Objetivo	Preparar para el centro de control una recomendación (acción, justificación, requisitos pendientes) que relacione el incidente con los recursos declarados y las restricciones.
Entradas	Incidente agrupado (evidencia, urgencia, incertidumbre), tabla de recursos declarados disponibles, restricciones operativas.
Salidas	`recommendation.proposed`: acción, recursos (sólo identificadores existentes), justificación con citas a eventos, requisitos pendientes, incertidumbre, `requires_approval: true`.
Herramientas	Lectura de recursos y políticas; modelo con salida estructurada; validador de referencias y citas.
Memoria	Ninguna propia: lee proyecciones.
Límites de autonomía	**No dispone de ninguna herramienta de despacho.** No asigna recursos críticos sin autorización ni inventa recursos; si faltan datos mínimos devuelve «información requerida» en vez de recomendar.
Manejo de errores	Referencia a un recurso inexistente o cita inválida → la recomendación se rechaza y se emite `recommendation.rejected`. Sin modelo: plantilla por tipo de incidente.
Terminación	Una recomendación por versión del incidente; caduca si el incidente cambia; se cierra con `approval.granted` o `approval.denied`.
Respaldo · modelo	Plantilla de acción. Modelo sugerido: `claude-opus-5-5` (bajo volumen y mayor riesgo).

Implementado (B4). Sólo la *plantilla*: una función pura, sin modelo, que propone una lista de acciones (rescate, bomberos, atención médica, despejar la vía; *una lista* porque un mismo incidente puede pedir rescate y ambulancia) y, para cada una, el recurso disponible más cercano de entre los declarados (`config/recursos.json`, **sintético**). Sólo las señales con evidencia verificable (un indicador marcado o una cita que es subcadena del texto de ese reporte) mueven recursos. Su salida —y la de cualquier proponente futuro, un modelo incluido— pasa por *una sola puerta*: lectura estricta del esquema y validación contra el incidente y los recursos *actuales* (recurso inexistente o no disponible, cita inventada, acción fuera del conjunto cerrado, texto libre donde van códigos, identificador que no corresponde a esta versión, falta de aprobación humana). El identificador es un hash de 16 hexadecimales de la *versión* (miembros y revisiones, urgencia, acciones y recursos): cambia si cambia el incidente y entonces la aprobación **caduca**. T13: 0 acciones o recursos inventados y 0 aprobaciones automáticas (`docs/resultados/b5.md`).

6.5 Integración con modelos de lenguaje

Principio rector: **el modelo es una mejora, no un requisito**. En el Escenario 2 los proveedores de internet están caídos, así que un diseño cuya función esencial dependa de una API en la nube fallaría justo cuando más se necesita. Por eso el acceso al modelo es un *puerto* con tres implementaciones intercambiables: nube, modelo local y reglas.

Tabla 15. *Asignación por defecto, proporcional al riesgo. Es una propuesta mía: el modelo de cada agente es configuración, no código.*

Agente	Qué hace el modelo	Modelo sugerido	Por qué
1 · Recepción	Sólo extrae tipo y referencias de lugar del texto libre.	`claude-haiku-5-5`	Alto volumen, tarea acotada, latencia baja.
2 · Triaje	Interpreta el texto, propone urgencia con citas.	`claude-sonnet-5-5`	Equilibrio de calidad y latencia en el camino crítico.
3 · Correlación	Adjudica pares «possible»; extrae afirmaciones para detectar contradicciones.	`claude-sonnet-5-5`	Sólo se invoca en casos dudosos: volumen bajo.
6 · Coordinación	Redacta la recomendación y su justificación.	`claude-opus-5-5`	Mayor riesgo y menor volumen: sólo incidentes priorizados.
4 y 5	No dependen del modelo.	—	Requisito del enunciado (agente 5) y de robustez.

Notas de integración con la API

- **Salida estructurada** con `output_config.format` y, además, validación propia del esquema. En `claude-sonnet-5-5` y `claude-opus-5-5` la herramienta forzada (`tool_choice` *any* / *tool*) devuelve error 400: usar salida estructurada.
- **Esfuerzo explícito.** En `claude-opus-5-5` el razonamiento no puede desactivarse y el esfuerzo por defecto es *medium*: fijarlo por agente.
- **Sin ajuste de muestreo.** En estos modelos `temperature` y afines no son configurables: el sistema trata el modelo como no determinista y por eso es idempotente y conserva ambos juicios.
- **Rechazos.** Revisar siempre `stop_reason` : un `refusal` (los clasificadores de seguridad pueden declinar una solicitud) es un fallo del agente y activa el respaldo.

Datos, costo y seguridad

- **Minimizar lo que sale a la nube:** redactar teléfonos y nombres, redondear la ubicación a una celda *geohash* si no se necesita el punto exacto. *Recomendación de diseño: en esta versión la redacción no está implementada y la ubicación no se envía al modelo (ver §8).*
- **Instrucciones estables primero:** un prompt de sistema largo y fijo por agente permite *prompt caching* y reduce costo y latencia.
- **Plazos cortos y reintentos acotados;** pasado el plazo, respaldo determinista.
- **Auditoría:** el evento guarda agente, versión, modo y modelo; la petición y la respuesta se archivan localmente.
- **Inyección de instrucciones:** el texto ciudadano es dato no confiable: se delimita, el agente de triaje no tiene herramientas con efectos y la salida se valida (ver §8).

6.6 Deduplicación, independencia de fuentes e incertidumbre

Es el corazón del Reto 4. Se separan tres problemas que suelen mezclarse.

Nivel 1 · Duplicado de transporte

El mismo mensaje llega varias veces porque el canal duplica o el teléfono reintenta. Se resuelve con la clave `(report_id, rev)` : el nodo ignora lo ya visto pero **siempre responde el ACK**, porque el ACK anterior pudo haberse perdido. Cuesta cero cómputo semántico.

Nivel 2 · Duplicado semántico

Varias personas reportan el mismo suceso con identificadores distintos. El agente 3 sigue tres pasos:

1. **Bloqueo.** Sólo compara reportes en la misma celda *geohash* o vecinas, dentro de una ventana de tiempo y con tipos compatibles; evita comparar todo con todo.
2. **Puntuación.** `score = w_d·K(d / (σ1+σ2)) + w_t·e^{−Δt/τ} + w_k·[mismo tipo] + w_s·sim(texto)`, con `K(x) = máx(0, 1−x)`. La distancia se mide *relativa a la precisión declarada* (σ) de cada reporte: dos puntos a 40 m con ±35 m de precisión son compatibles; a 400 m no. Valores iniciales: `w = (0,40; 0,15; 0,15; 0,30)`, `τ = 30 min`.
3. **Decisión tri-estado.** *same* si `score ≥ θ_hi` (inicial 0,80); *possible* si `θ_lo ≤ score < θ_hi` (inicial 0,55); *different* en otro caso. Sólo *same* agrupa; *possible* se sugiere al coordinador. Los umbrales se calibran con T07 priorizando **no fusionar casos distintos**.

Nivel 3 · Rumor frente a corroboración

La pregunta del Reto 4 es cómo distinguir un rumor repetido mil veces de una emergencia corroborada por fuentes independientes. La respuesta es que **contar mensajes no mide confianza; contar fuentes independientes sí**.

- Cada dispositivo cuenta una vez. Los textos casi idénticos de dispositivos distintos, cercanos en el tiempo, forman una *familia de copias* que cuenta como una sola fuente.
- La confianza combina las fuentes independientes con una regla de tipo *noisy-OR*: `P(real) = 1 − Π (1 − p_i·w_i)`, donde `p_i` es la credibilidad base de la clase de fuente y `w_i` su factor de independencia.
- **Ejemplo ilustrativo** (valores no calibrados): un rumor por radio repetido 1 000 veces es una sola familia con `p = 0,2` → `P = 0,20`; cinco reportes de la app con ubicación, de dispositivos distintos y con texto propio (`p = 0,6`) → `P = 1 − 0,4^5 ≈ 0,99`.

Contradicciones e incertidumbre

Las contradicciones («puente transitable» frente a «puente colapsado») **se conservan y se muestran**; nunca se resuelven en silencio, porque pueden cambiar la respuesta. La incertidumbre es un campo del esquema con códigos explícitos — `ubicacion_ausente`, `ubicacion_imprecisa`, `texto_truncado`, `fuente_unica`, `fuente_no_independiente`, `contradiccion`, `dato_antiguo`, `baja_confianza_modelo` — que el tablero convierte en insignias. Sin modelo de lenguaje, la detección de contradicciones queda «no evaluada» y así se indica, en lugar de afirmar que no las hay.

Un juez de modelo para el Nivel 2: medido y no adoptado

El puntaje léxico no ve que «me cayó el techo encima» y «quedamos encerrados» son lo mismo. Se construyó (`link_judge`, función pura) un juez que le pide al modelo veredictos *same / possible / different* entre un reporte nuevo y hasta 6 candidatos anteriores. **El modelo propone, el código dispone:** un par fuera de la ventana de tiempo, de tipo incompatible o más lejos que $\sigma_1+\sigma_2$ no se une nunca; cada cita debe ser un fragmento literal de su texto o se descarta la respuesta entera; ante la duda el modelo no une; y aplicar los juicios es una función pura que no depende del orden ni del nodo (replicable como un evento más). Medido una vez contra la API real, **el criterio fijado de antemano no se cumple** y el nodo no lo usa: el recall de pares sube de 0,8 % a 4,4 % y la corroboración de 5 fuentes se reúne en 4 con confianza 0,60 → 0,97, pero la falsa fusión sube de 1,22 % a 2,21 %. Resultados y límites en el capítulo 9 y en `docs/resultados/reto4-juez.md`.

Qué se implementó en B3 y qué se midió

Los agentes 1, 2 y 3 son **funciones puras** en `packages/agents`; el nodo sólo los invoca. El agente 2 combina banderas y texto (las señales del texto llevan una cita que es subcadena exacta del original; la negación se entiende dentro de la oración; el texto es dato no confiable). El agente 3 puntúa pares con los pesos de arriba y los umbrales calibrados con T07 (*same* desde 0,85, o 0,90 si falta la ubicación; los iniciales del PDF eran 0,80 y 0,92), y **el grupo se calcula al leer**, como función pura de los reportes ya fusionados: no depende del orden ni de qué nodo

vio qué (dos nodos aislados que luego replican muestran el mismo grupo) y un reporte cuya ubicación llega en una revisión se une a su grupo. Los eventos `incident.linked` son rastro de auditoría, no la fuente del grupo. El modelo de lenguaje es opcional: un puerto con respaldo por reglas, usado sólo en el triaje.

Hallazgos de la medición que cambian el diseño de arriba (cifras en `docs/resultados/b3.md`):

- **«Falta» no es «distinto».** Con los pesos del PDF, un par de SOS de un toque (sin texto) no puede pasar de 0,70 y nunca es *same*. Y una similitud con una sola palabra de contenido («no hay atrapados» frente a «estoy atrapado») no es evidencia. Se ponderó la similitud por la *masa de evidencia* (`mín(1, palabras compartidas / 5)`).
- **Un agrupamiento determinista no separa un duplicado de un suceso vecino casi idéntico.** Cumplir «falsa fusión ≤ 1 %» exige un recall de pares cercano a cero (0,4 %), y aun así **con semillas nunca vistas una de cinco supera el 1 %** (2,9 %): la cifra es inestable. Con reportes *sin texto*, que es lo que produce la app, no agrupa ningún par verdadero; *sí sugiere* el 15,7 % con 97 % de precisión (21,4 % y 92 % con la mezcla de reportes con y sin texto): es la franja *possible* que el PDF reserva al modelo y al coordinador.
- **Un reporte de poca evidencia espacial no puede hacer de puente.** En el banco de pruebas un reporte sin ubicación con un texto común unió reportes de puntos distintos y salió *un grupo de 1 705 reportes que abarcaba 27 km*. Un reporte sin ubicación o impreciso puede sumarse a un grupo, pero no unir dos grupos que ya tienen un reporte localizado con precisión; el vínculo frenado queda como sugerencia.
- **El coste de leer la cola no era menor.** La primera versión tardaba 2,4 s con 1 000 reportes y *cuatro minutos* con 10 000, en el mismo hilo que atiende los ACK. Tras indexar (rejilla de ≈ 500 m, política de bloqueo sin ubicación por contenido idéntico) y mover el cálculo a otro *isolate* con caché por número de eventos, 10 000 reportes tardan ≈ 7,5 s *en segundo plano* y el hilo del nodo recupera el control en ≈ 0,1 s (copiar los eventos al isolate es lo que queda en el hilo principal). Si el cálculo es lento y el log cambió, se sirve la versión anterior marcada como vieja y con su antigüedad —con un **tope de 5 s**: pasado ese plazo se espera al dato fresco, para que un P1 recién llegado no quede oculto—, y el tablero muestra el aviso («vista de hace N s… puede faltar algún reporte reciente»). El pipeline añade 50–120 ms por trama con hasta 10 000 reportes ya en el log.
- **Escalar a revisión humana no es gratis.** Contar la revisión como acierto sube el recall de P1 al 87,7 %, pero escala el 26 % de todos los reportes y sólo el 23 % de ellos era de verdad P1 o P2: más de tres de cada cuatro avisos serían falsas alarmas. Y el tablero aún no muestra ese aviso.

6.7 El tablero del coordinador

El Reto 6 pregunta cómo reducir la carga cognitiva del coordinador sin ocultarle información importante. La respuesta propuesta es **resumir con contadores honestos**: vista corta por defecto, con lo que queda fuera siempre a la vista.

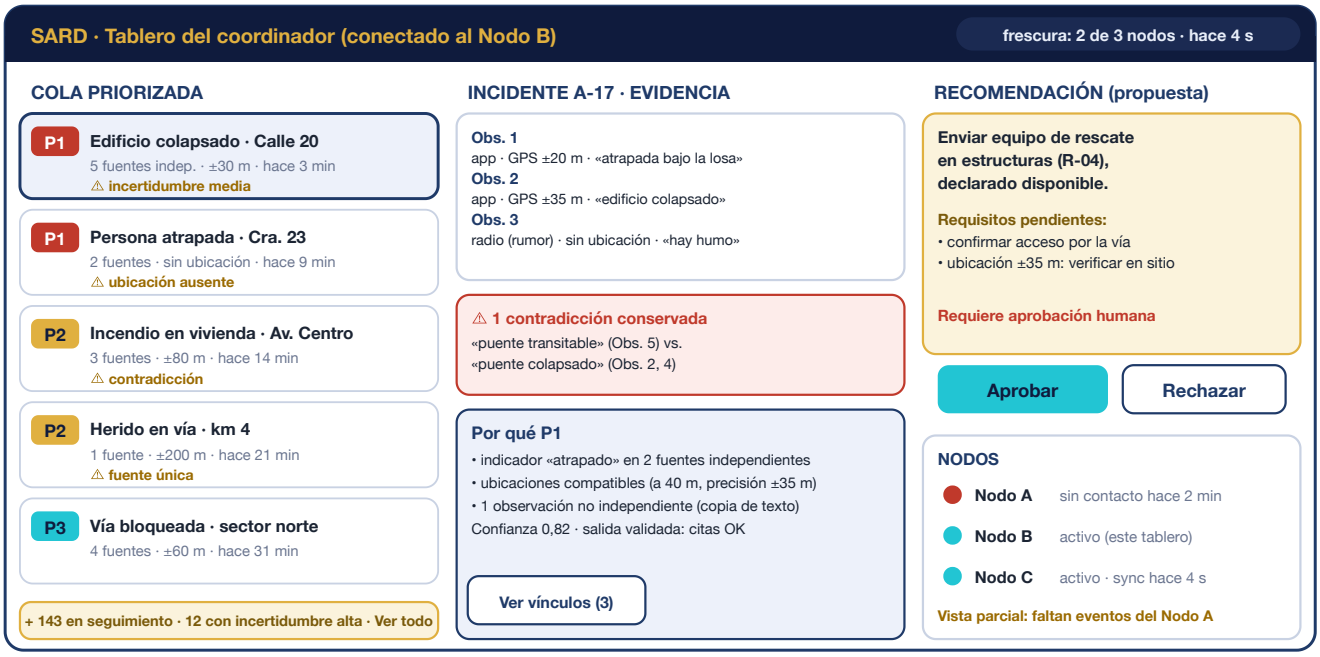


Figura 7. *Tablero del coordinador (wireframe con datos ilustrativos). Lista priorizada, evidencia, contradicciones, recomendación con aprobación y estado de los nodos.*

- **Unidad = incidente, no mensaje.** Mil reportes se presentan como decenas de incidentes con sus fuentes.
- **Lista corta con contadores.** Una franja fija indica cuántos incidentes quedan en seguimiento y cuántos tienen incertidumbre alta. **Ningún P1 queda fuera de la vista inicial.**
- **Por qué.** Cada incidente explica su prioridad con las señales y citas del agente 2 y enlaza sus observaciones originales.
- **Siguiente acción sugerida y aprobación.** La recomendación del agente 6 aparece con sus requisitos pendientes; aprobar o rechazar queda registrado.
- **Frescura y funcionamiento sin Internet.** Se indica qué nodos están sincronizados y cuándo; si faltan eventos, el tablero declara la «vista parcial». Sus recursos los sirve el nodo y el mapa es esquemático o con teselas precargadas.

Qué se implementó en B4–B5 y qué se midió

El tablero es una aplicación de Flutter Web que sirve el propio nodo (`--web-root`). **El incidente es la unidad**: el nodo lo calcula como *proyección de lectura* (una función pura de los reportes fundidos, las decisiones y los recursos declarados), en el mismo pase que la cola, y sirve un **resumen** por incidente y un **detalle** bajo demanda.

- **Vista corta y contadores honestos.** Por omisión: *todos* los P1 y P2, los 20 P3/P4 con información más recientes y *una* fila agregada de «N SOS sin detalle» (lo que más emitirá la app real). Los contadores son sobre el total real (por urgencia, incertidumbre alta, a revisar, SOS sin detalle y «en seguimiento fuera de la vista»); una barra de triaje P1–P4 proporcional que además filtra con un clic. La vista corta ignora la paginación a propósito: paginarla podría dejar un P1 fuera.
- **Por qué.** Cada incidente muestra las señales con su evidencia (indicador marcado o cita exacta del texto), las observaciones originales y la incertidumbre; «*contradicciones entre reportes: no evaluadas*», porque la correlación no usa modelo.
- **Recomendación y aprobación.** La recomendación del agente 6 aparece con sus requisitos pendientes («antes de aprobar»). Aprobar o rechazar se *registra siempre* como evento, con el nombre y la versión que la persona vio; justo antes de registrar el tablero vuelve a leer el incidente y, si la recomendación cambió, **no registra** y muestra la nueva. «Vigente», «caduca» y «discrepante» se derivan al leer. Dos decisiones contradictorias de la

misma versión se conservan: manda el rechazo y se marca la discrepancia hasta que una persona sustituya *explícitamente* una de ellas. El nombre es texto libre: el código de acceso compartido (`--coordinator-token`, opcional) abre la puerta del tablero pero **no identifica** a quien aprueba.

- **Frescura y vista parcial.** Sin conexión, vista vieja (con su antigüedad), respuesta inválida y «vista parcial» (con el motivo y el nodo) se dicen con un aviso y se conserva la última vista válida. Un nodo recién reiniciado no se muestra «sincronizado». Un tablero que sondea sin cambios recibe `304` (`ETag` que distingue cada arranque).

Tabla 16. *P1 fuera de la vista inicial: 0 en las cinco semillas (criterio cumplido); P1 a 0 clics, su detalle a 1.*

T14 estructural · 1 000 reportes, 5 semillas nuevas	Elementos a revisar para cubrir todos los P1
(a) mensajes en orden de llegada	≈ 996
(b) reportes ordenados por urgencia, sin agrupar	≈ 184
(c) incidentes de la vista corta del tablero	≈ 183
Techo con agrupamiento perfecto (verdad del generador)	≈ 20

Hallazgo que cambia la expectativa del diseño. La mejora de ≈ 996 a ≈ 184 viene de *ordenar por urgencia*; **no** de agrupar (184 → 183), porque el agrupamiento determinista casi no fusiona (B3). Un agrupamiento perfecto llevaría la cifra a ≈ 20: esa diferencia queda para el modelo y para la persona. Además, con datos sintéticos de ≈ 20 % de P1, «todos los P1 y P2» ya pesan ≈ 4 MB con 10 000 reportes: el criterio «ningún P1 fuera» manda, y una mezcla real tendría menos. **No** se hizo la prueba con coordinadores (tiempo y errores) que pide el documento; verificado en Chrome de escritorio y un emulador, sin lector de pantalla.

7 Protocolo de comunicación SARD-WIRE v0

Este capítulo respalda el entregable 5: esquema del mensaje, reintentos, confirmaciones, deduplicación y estrategia de sincronización. Es el contrato entre la app, los nodos y el simulador, y se define de forma que **cualquier lenguaje pueda implementarlo** verificándolo contra vectores de prueba.

7.1 Objetivos y supuestos del canal

El enunciado no fija un canal ni un tamaño máximo. El protocolo se diseña para el peor caso razonable: una trama completa del caso normal debe caber en un solo paquete de un canal de baja capacidad. Los valores siguientes son magnitudes de referencia habituales y deben verificarse para el canal que finalmente se use.

Tabla 17. *Referencias de carga útil por canal. Se diseña para que el caso normal quepa en 100 bytes y se fragmenta sólo cuando hace falta.*

Canal (ejemplo)	Carga útil de referencia	Consecuencia de diseño
HTTP/Wi-Fi local	Sin límite práctico	Es el canal de la demostración; las tramas viajan igual que en los demás.
SMS	140 bytes por mensaje	El SOS (33 B) y una revisión con texto corto (123 B) caben en un mensaje.
Bluetooth LE 5 (publicidad extendida)	Del orden de 250 bytes por paquete	Cabe el SOS y la revisión con texto sin fragmentar.
Radio de baja potencia (tipo LoRa)	Del orden de 50 a 250 bytes según configuración	El SOS cabe siempre; los textos largos se fragmentan.

7.2 Formato de la trama

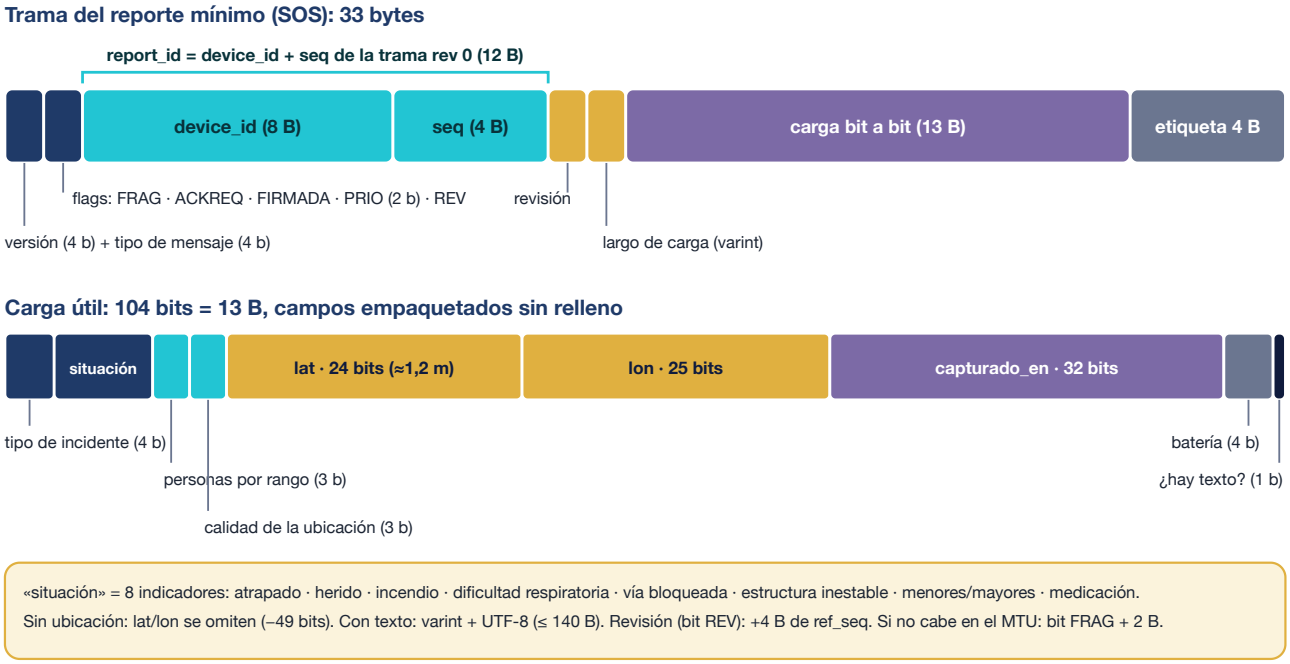


Figura 8. *Trama del SOS mínimo (33 bytes) y empaquetado bit a bit de su carga útil (104 bits).*

Tabla 18. *Campos de la trama* `REPORT`*. Se usa el orden de red (big-endian).*

Campo	Tamaño	Descripción
versión · tipo	1 B	4 bits de versión del protocolo y 4 bits de tipo de mensaje.

Campo	Tamaño	Descripción
flags	1 B	Bit 7 `FRAG`, bit 6 `ACKREQ`, bit 5 `FIRMADA`, bits 4–3 `PRIO` (0 = P1 … 3 = P4, sólo pista del teléfono), bit 2 `REV` (la trama es una revisión y lleva `ref_seq`), bits 1–0 reservados.
`device_id`	8 B	Número aleatorio de 64 bits generado en la instalación; no identifica al teléfono.
`seq`	4 B	**Contador de tramas del dispositivo:** cada trama enviada, sea un reporte nuevo o una revisión, consume un valor. Es lo que acusa el ACK, de modo que cada revisión se confirma por separado.
`rev`	1 B	Revisión del reporte (0 = SOS inicial). Gana la revisión más alta.
`ref_seq`	0 ó 4 B	Sólo si `REV`: el `seq` de la trama de revisión 0 del reporte. Así, `report_id` = (`device_id`, `seq` de la trama rev 0) identifica al reporte en todas sus revisiones y es la **clave de idempotencia** junto con `rev`.
fragmento	0 ó 2 B	Sólo si `FRAG`: índice y total de fragmentos.
largo	1–2 B	Largo de la carga útil (varint).
carga útil	13 B + texto	Campos fijos empaquetados sin relleno (104 bits) y texto opcional (varint + UTF-8, ≤ 140 B).
etiqueta	4 B	HMAC-SHA256 truncado sobre cabecera y carga. En v0, con una **clave de evento precompartida** (ver §7.6).

Carga útil fija. Tipo de incidente (4 b), ocho indicadores de situación (8 b: atrapado, herido, incendio, dificultad respiratoria, vía bloqueada, estructura inestable, menores o mayores, medicación), personas por rango (3 b: 0 no indicado, 1, 2–5, 6–20, 21–100, más de 100; 6 y 7 reservados, se leen como «no indicado»), calidad de la ubicación (3 b), latitud y longitud cuantizadas (24 + 25 b, resolución cercana a 1,2 m), hora de captura en segundos (32 b), batería en 16 pasos (4 b) y un bit que indica si hay texto. Si la ubicación no está disponible (calidad 0 o manual), latitud y longitud se omiten.

Vector de prueba. Con la clave de prueba `clave-de-ejemplo`, el SOS de ejemplo del repositorio (`docs/src/tools/frame_size.py`) se codifica en estos 33 bytes. Una revisión añade 4 bytes de `ref_seq`:

```
01 60 9a3c51e07b2d44f1 00000011 00 0d 3c04a1cd82929348 6d4f3b6462 2bf36cc5
ver|tipo · flags · device_id · seq · rev · len · carga (13 B) · etiqueta
```

7.3 Tipos de mensaje

Tabla 19. *Tipos de mensaje. Los mensajes entre nodos no tienen la restricción de bytes del enlace con el teléfono.*

Tipo	Nombre	Sentido	Contenido
`0x1`	REPORT	Teléfono → nodo	Reporte inicial o revisión (trama descrita arriba).
`0x2`	ACK	Nodo → teléfono	`device_id` (8 B), `ack_seq` (4 B: la trama que este ACK responde), `bitmap` (4 B: el bit *i* indica que el nodo guarda además la trama `ack_seq − 1 − i`) y etiqueta. **22 bytes** en total. Al contar tramas y no reportes, confirma también cada revisión. **Invariante:** un ACK sólo cubre tramas que el nodo que lo emite tiene guardadas.
`0x3`	STATUS	Nodo → teléfono	Confirmación de segundo nivel que mueve el reporte a «Recibido por SARD»: indica que está en la cola priorizada **y**, si hay dos o más nodos vivos, que existe en al menos dos (número de réplicas). Cuando el nodo ya sabe quiénes son sus pares vivos, puede añadirlos a esta respuesta.
`0x4`–`0xF`	Reservados	—	Evolución del protocolo (la versión va en la propia trama).
—	SYNC_HAVE / SYNC_EVENTS	Nodo ↔ nodo	Vector de versiones y lotes de eventos (ver §6.2). Viaja por un canal sin restricción de tamaño; no usa esta trama compacta.

Tipo	Nombre	Sentido	Contenido
—	HEARTBEAT	Nodo ↔ nodo	Latido para el detector de fallos del agente 5.

7.4 Fiabilidad: confirmaciones, reintentos y fragmentación

- **Confirmaciones.** El nodo escribe `report.received` de forma durable y entonces responde un *ACK selectivo con ventana hacia atrás*: un paquete de 22 B acusa la trama recibida y hasta 32 anteriores que ese nodo también guarda. No es acumulativo desde 1 porque, con varios nodos y *failover*, ningún nodo ve todas las tramas de un teléfono; un ACK acumulativo dejaría sin acusar para siempre todo lo que supere la ventana. También responde ACK ante un duplicado, porque el ACK original pudo haberse perdido.
- **Reintentos.** El temporizador de cada trama vence y se reprograma con espera exponencial y *variación completa* (ver §5.5). El teléfono reintenta contra cualquier nodo conocido, no sólo contra el último. En v0 esa lista es una configuración estática (las direcciones de los nodos o del proxy de canal); como extensión, cada `STATUS` puede anunciar los pares vivos.
- **Ráfagas.** En una ventana abierta se envía una ventana deslizante de hasta 8 tramas sin esperar un ACK por trama; al cerrarse la ventana, la espera vuelve a ser larga.
- **Desorden y duplicados.** No se exige orden de llegada. El receptor aplica la revisión más alta por reporte; una revisión antigua que llega tarde se guarda pero no sobrescribe.
- **Fragmentación.** Sólo cuando la carga no cabe en el MTU del canal: bit `FRAG` más (índice, total). Cada fragmento repite `report_id` y `rev`; cada fragmento consume su propio `seq` y se acusa con el mismo ACK selectivo; el receptor reensambla con un plazo. En v0 la fragmentación está definida pero no implementada (el decodificador rechaza `FRAG`). Un reporte incompleto no se descarta: se guarda como parcial.
- **Prioridad y envejecimiento.** El campo `PRIO` ordena el envío, pero una cuota reserva posiciones para lo más antiguo: la prioridad nunca hace desaparecer un mensaje.
- **Vencimiento.** Al cumplirse el TTL del reintento activo el mensaje pasa a envío oportunista de baja frecuencia; **nunca se elimina** del dispositivo.

Tabla 20. *Parámetros iniciales. Son valores de arranque míos, no requisitos; las pruebas los ajustan.*

Parámetro	Valor inicial	Se calibra con
Base y factor de espera de reintento	2 s, ×2, con variación completa	T03
Tope de espera entre reintentos	300 s	T04
Ventana deslizante de envío	8 tramas	T03, T04
Cuota contra la inanición	1 de cada 5 posiciones del lote	T11
Plazo de reensamblado de fragmentos	300 s	T03
TTL del reintento activo	Configurable (propuesta: 6 h)	Política operativa
Reportes nuevos por dispositivo y hora	Propuesta: 20 (los SOS no cuentan)	T05, T08

7.5 Deduplicación en el transporte y revisiones

- **Clave.** `(report_id, rev)`, con `report_id` = (`device_id`, `seq` de la trama rev 0), es única **en el log de cada nodo**: reenviar la misma trama a un mismo nodo cualquier número de veces deja el mismo estado (ver T05). **Entre nodos no:** si un nodo guarda una trama y su ACK se pierde, el teléfono la reintenta contra otro nodo antes de que se replique, y ambos escriben su recepción (en el recorrido H1 medido: 3 de 72, 4 de 72 y 19 de 240 pares). Esos eventos no se descartan —saltarlos abriría huecos en el vector de versiones y la replicación no convergería—; la cola priorizada los funde por reporte. El ACK se refiere a la *trama* (`seq`), por lo que distingue cada revisión.

- **Enriquecer en vez de duplicar.** Cuando el ciudadano añade personas o texto, la app envía una trama nueva (con su propio `seq`) que apunta al mismo reporte mediante `ref_seq` y lleva `rev + 1`. El nodo funde las revisiones *campo a campo*: el último valor de cada campo gana, un campo ausente conserva el anterior.
- **Costo de una revisión.** Sin texto ocupa 37 bytes (los 33 del SOS más 4 de `ref_seq`); con un texto de 85 caracteres ocupa 123 bytes. Una optimización futura son tramas «delta» que sólo lleven los campos cambiados.
- **El mismo ciudadano reporta dos veces** (dos `report_id`): es un duplicado semántico y lo resuelve el agente 3, no el transporte.

7.6 Seguridad, integridad y abuso

- **Integridad.** Una etiqueta de 4 bytes detecta corrupción y suplantación casual (probabilidad de acierto aleatorio de 1 entre 2^{32} por intento).
- **Provisión de la clave (supuesto de v0).** Un teléfono que se instala o se activa sin conexión no puede haber registrado una clave propia. Por eso, en la demostración, la etiqueta usa una **clave de evento precompartida** que se siembra en la app y en los nodos al preparar el entorno. Limitación: quien tenga la app puede falsificar tramas; protege contra corrupción y errores, no contra un adversario. Para producción: registro de una clave por instalación en el primer contacto con conexión, o firma asimétrica.
- **Límite honesto.** Esta etiqueta *no* protege contra un atacante dedicado que pueda intentar millones de veces. Para producción se usaría una etiqueta más larga o una firma asimétrica, a costa de más bytes por trama.
- **Abuso.** Presupuesto por dispositivo que degrada (no rechaza; sólo se omite el modelo), reputación que reduce el *peso de corroboración* (no bloquea un SOS), y detección de copias de texto.
- **Privacidad.** Sin identidad obligatoria; `device_id` es un número aleatorio, no un identificador del teléfono; los canales con Internet usan TLS.

7.7 Qué se midió

Las cifras del documento provienen de una calculadora de referencia que empaqueta la trama propuesta y la compara con dos líneas base JSON: una con **claves descriptivas y hora ISO 8601** (lo habitual en una API) y otra **compacta** (claves de una letra, hora en segundos y banderas como entero). El titular de «7 veces menor» depende de la primera; frente al JSON ya optimizado la ventaja es de unas 3 veces. No es una medición de red: es el tamaño del mensaje.

Tabla 21. *Tamaños medidos con* `python3 docs/src/tools/frame_size.py`*. Un SOS entregado y confirmado cuesta 55 bytes en total (33 B de ida y 22 B de vuelta), sin contar reintentos.*

Caso	Trama binaria	JSON descriptivo	JSON compacto
SOS mínimo (revisión 0)	33 B	239 B (7,2 ×)	104 B (3,2 ×)
Revisión sin texto	37 B	239 B (6,5 ×)	111 B (3,0 ×)
Revisión con texto de 85 caracteres	123 B	334 B (2,7 ×)	203 B (1,7 ×)
ACK selectivo con ventana	22 B	—	—

7.8 Canal del despachador (fuera de la trama)

El Escenario 2 habla de «rumores capturados por radio de onda corta y mensajes a medio terminar». La trama binaria de este capítulo es de la app; lo que un despachador oye lo registra una persona en el tablero, por `POST /api/dispatch` (JSON, sólo `application/json`, hasta 8 KiB). Es información de **segunda mano** y recorre el mismo camino que un aviso de la app.

Tabla 22. *Cuerpo de* `POST /api/dispatch`. *Cualquier otro campo se rechaza (400) diciendo cuál.*

Campo	Significado
`id`	De 8 a 64 caracteres, lo genera el tablero. Reintentar con el mismo `id` y el mismo contenido **no duplica** (200, `duplicate: true`); el mismo `id` con otro contenido es un **409**, no se descarta en silencio.
`text`	Lo que se oyó; hasta 500 *caracteres* (no bytes). Es dato no confiable, como cualquier texto ciudadano.
`medium`	`radio`, `telefono` o `presencial`: sólo una etiqueta para quien lee; los tres comparten la clase de fuente `radio`.
`origin`	Opcional: de quién viene la información. **Decide la independencia**: el mismo origen es una fuente; varios orígenes con el mismo texto son copias y también cuentan como una.
`who`	Quién registra (declarado, no autenticado).
`flags`, `people`	Opcionales, con el mismo significado que en la app (por nombre y por código de rango). Con sólo texto el triaje por reglas acierta menos.

- **El evento.** Se escribe un `report.received` con `channel: dispatcher`, `source_class: radio` (credibilidad base 0,2, supuesta y sin calibrar), el medio, el origen y quién lo registró. Lleva las mismas claves que uno de trama, con nulos en las propias de la trama (`frame_seq`, `ref_seq`, `battery_q`): ningún lector tropieza. Se replica entre nodos como cualquier evento.
- **Sin ubicación.** El agente 1 no la inventa y no hay campo para ella; como consecuencia el agente 6 no recomienda («falta saber la ubicación») y el coordinador debe resolver el dónde.
- **La lista lo dice.** Un incidente hecho sólo de informes de despachador se marca «De segunda mano» en su fila; uno mixto, «Incluye informe de despachador».
- **La hora es la del nodo al registrar**, no la de cuando se oyó; el tablero lo dice.
- **El modelo de lenguaje no ve este texto** aunque esté activado: por radio es más probable que traiga nombres, teléfonos y direcciones, y hoy no se redacta nada (§8).
- **Autorización.** Con `--coordinator-token` exige ese código (denegado por omisión bajo `/api/`); sin él, quien llegue por red puede registrar. El código no identifica a quien registra.
- **Límite.** No agrupa lo que no se parece: varios orígenes que cuentan el mismo suceso con otras palabras siguen siendo incidentes distintos (T08 de punta a punta). Resultados y límites: `docs/resultados/despachador.md` (T16).

8 IA responsable y seguridad

El objetivo 9 del enunciado pide «prevenir invenciones, decisiones injustas, sesgos y acciones autónomas peligrosas». La estrategia no es pedirle al modelo que se porte bien, sino **diseñar el sistema para que no pueda portarse mal**: limitar lo que los agentes pueden hacer, validar lo que producen y dejar todo registrado. Cada control tiene una prueba que lo verifica.

Tabla 23. *Riesgos de IA y seguridad, control de diseño que los contiene y prueba que lo verifica.*

Riesgo	Control de diseño	Prueba
Invención de datos (ubicación, cifras, recursos)	El esquema admite `null` explícito para lo desconocido. Un validador exige que cada cita sea subcadena del texto original o un campo existente, y que cada recurso citado exista en la tabla declarada. Lo inferido se marca `inferido`.	T13
Acciones autónomas peligrosas	Los agentes **no tienen herramienta de despacho**; sólo emiten eventos de propuesta. Toda recomendación exige `approval.granted` de una persona identificada.	T13
Inyección de instrucciones en el texto ciudadano	El texto es dato no confiable: se delimita en el prompt, el agente de triaje no tiene herramientas con efectos, la salida se valida por esquema y el *piso por reglas* no puede ser rebajado por el modelo.	T13
Decisiones injustas y sesgos	La urgencia no usa propiedades del dispositivo ni del usuario. El recall de P1 se desglosa por subgrupo y etiqueta (con y sin texto, con y sin ubicación, ubicación imprecisa, parafraseado, con faltas, reloj desfasado; ver `docs/resultados/b3.md`), y la disparidad se reporta tal cual: 66,1 % sin texto frente a 85,9 % con texto. **No se midieron** texto corto frente a largo ni otros idiomas o dialectos. El aviso de que **el silencio no es seguridad** (un mapa de cobertura que señale las zonas sin reportes) está **pendiente**: el mapa del tablero se pospuso (capítulo 10) y hoy el tablero no distingue «sin reportes» de «sin problemas».	T06 por subgrupo (parcial)
Indisponibilidad o degradación del modelo	Respaldo determinista para cada agente; el evento indica el modo usado (`rules` o `rules+llm`) y se mide la diferencia de calidad.	T12
Opacidad de las decisiones	Cada evento guarda el nodo de origen, la versión del agente (`agent_version`), el modo (`rules` o `rules+llm`), la evidencia con sus citas y los eventos de entrada. **El nombre del modelo no se guarda en el evento.** El tablero muestra el «por qué» con señales y citas. El log inmutable es el registro de auditoría.	T15
Pérdida de contexto al fusionar	Los grupos son vistas reversibles con vínculo a cada observación original; las contradicciones se conservan.	T07
Privacidad de datos de emergencia	Sin cuenta obligatoria. A servicios externos **no se envían** la ubicación ni el identificador del dispositivo (sí el texto libre de la app, sin redactar: ver el recuadro de abajo; el texto del canal del despachador, §7.8, no se envía nunca). La ubicación **no se redondea**: la trama la cuantiza a ≈ 1,2 m. **Pendientes:** redacción de datos personales, política de retención y acceso por persona; el acceso al tablero y a la réplica puede exigir un código compartido opcional, que no identifica a quien entra.	Revisión (parcial)
Abuso y spam	**Presupuesto de trabajo caro por dispositivo que degrada y no rechaza**: una trama de un dispositivo pasado de presupuesto se guarda y se acusa igual, y sólo se omite la consulta al modelo (la clave de integridad es compartida; un rechazo permitiría bloquear el SOS de una víctima falsificando su identificador). Peso de corroboración por independencia de fuentes; detección de copias de texto.	T05, T08

ESTADO REAL EN B3: PRIVACIDAD DEL MODELO DE LENGUAJE

Con `--llm on` el nodo envía a un servicio externo (la API de Anthropic) el **texto libre del ciudadano**, tal cual, junto con las banderas marcadas. **No se redacta ningún dato personal** (nombres, teléfonos, direcciones escritas a mano) *antes* de enviarlo: eso queda **pendiente**. No se envían la ubicación ni los identificadores del dispositivo, y **el texto de los informes de despachador tampoco** (por radio es más probable que traiga nombres y teléfonos). Está **apagado por defecto**; sin clave o sin red el nodo sigue con las reglas. La ruta del modelo se verificó contra la **API real** el 9 de noviembre de 2026 (`docs/resultados/modelo-real.md`): en 750 reportes sintéticos hubo 1 salida inválida y ninguna llamada por encima de los 8 s del plazo; el nodo envía por omisión `claude-sonnet-5-5` con `effort low` (lo medido), y cada vez que se cae al respaldo queda contado por causa en `/api/health`.

UN PRINCIPIO QUE CONVIENE DEFENDER ANTE EL JURADO

Ante la incertidumbre, el sistema **sube el caso a revisión humana; nunca lo baja de prioridad**. Un falso positivo cuesta atención del coordinador; un falso negativo puede costar una vida. Por eso las métricas de triaje separan el *recall* de los casos críticos de la exactitud global (ver §9.3).

9 Estrategia de pruebas y métricas

Este capítulo respalda el entregable 6 («métricas, escenarios ejecutados, errores observados, limitaciones y resultados de las pruebas de estrés») y las pruebas reproducibles que pide el objetivo 10. La idea central es que **el banco de pruebas es un componente del producto**, no un añadido de última hora.

9.1 El banco de pruebas

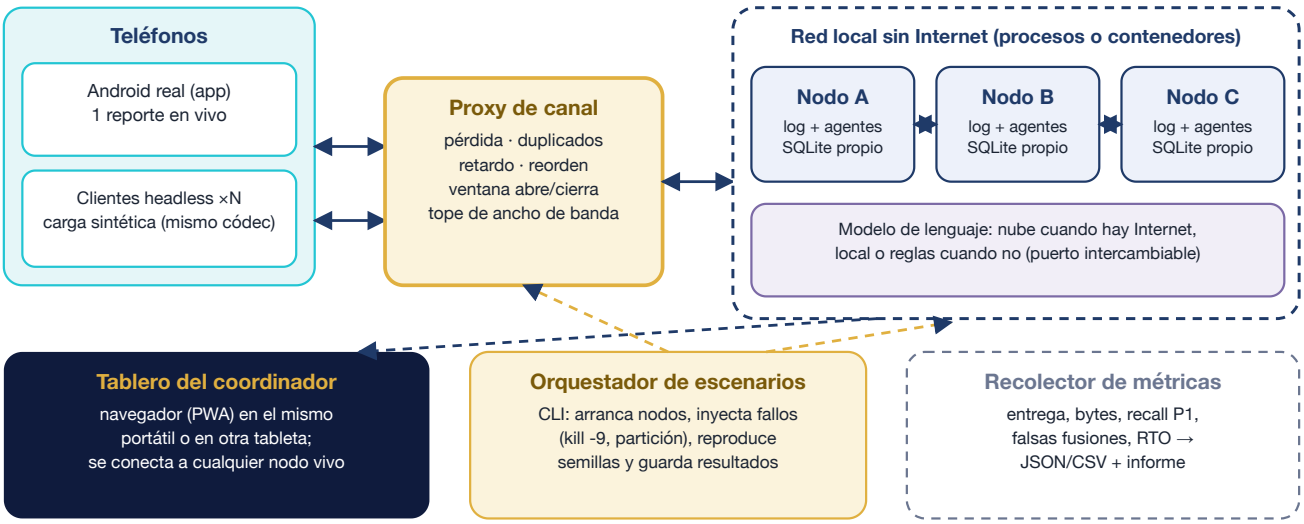


Figura 9. *Despliegue de la demostración y de las pruebas. Todo corre en una red local sin Internet; el modelo de lenguaje es opcional.*

Generador de datos sintéticos

- Crea incidentes con **verdad de referencia**: a qué incidente pertenece cada reporte y qué urgencia real tiene.
- Aplica ruido controlado: ubicación imprecisa o ausente, texto truncado, duplicados exactos, copias de rumores, contradicciones y *pares adversariales* (incidentes cercanos pero distintos).
- Es determinista con una semilla: el mismo escenario produce los mismos datos.

Simulador de canal e inyector de fallos

- Pérdida (independiente o en ráfagas), duplicación, reorden, retardo con variación, tope de ancho de banda y **ventanas que se abren y cierran**.
- `kill -9` de un nodo, partición de red, reinicio y relojes desfasados.
- Un orquestador de escenarios lee un archivo YAML y lo ejecuta de forma reproducible.

```yaml
# scenarios/s2-kill-node-a.yaml (ejemplo de escenario reproducible)
id: s2-kill-node-a
seed: 42
topology: { nodes: [A, B, C], sync_interval: 5s }
channel:
  loss: 0.40            # pérdida; alternativa: modelo de ráfagas
  duplicate: 0.05
  reorder: 0.10
  delay_ms: [200, 3000]
  window: { open: 3s, closed: 57s }
load:
  generator: synthetic-v1
  incidents: 80
  reports: 1000
  noise: { location_m: 40, truncated_text: 0.10, copies: 0.20, adversarial_pairs: 20 }
faults:
  - { at: 120s, action: kill9,   target: A }
  - { at: 300s, action: restart, target: A }
assert:
  - acked_reports_lost == 0
  - p1_recall >= 0.95
  - false_merge_rate <= 0.01
  - convergence_time <= 60s
```

El modelo de lenguaje y la reproducibilidad. Como el modelo no es determinista, las pruebas tienen dos modos: *reglas* (totalmente reproducible) y *modelo con respuestas grabadas* (se graba una ejecución real y se reproduce). La calidad con modelo real se mide aparte y se reporta como una diferencia respecto al modo de reglas (T12).

9.2 Catálogo de pruebas

Las metas son **sugerencias mías a calibrar**, no requisitos del enunciado; sirven para convertir cada propiedad en algo falsable.

Tabla 24. *Catálogo de pruebas. Cada una produce datos que alimentan el informe del entregable 6.*

ID	Qué verifica	Escenario (parámetros propuestos)	Métrica	Meta sugerida
T01	Persistencia ante cierre brusco	Crear 100 reportes; matar la app, reiniciar el teléfono y activar modo avión entre medias.	Reportes «Guardado» conservados	100 %
T02	Presupuesto de bytes	SOS mínimo y revisión con texto; con y sin ubicación.	Bytes por reporte; bytes por reporte confirmado	SOS ≤ 40 B; con texto ≤ 160 B
T03	Entrega bajo pérdida	Pérdida 0, 30, 60 y 90 % con duplicados y reorden.	Razón de entrega; latencia p50/p95 de P1	Curva reportada; P1 ≥ 99 % si hay capacidad
T04	Ventanas breves	Ventana de 3 s cada 60 s.	SOS entregado en la primera ventana	≥ 95 %
T05	Idempotencia	Duplicación masiva y reintentos agresivos.	Duplicados por reintento dentro de un nodo; duplicados en la cola priorizada; recepciones repetidas entre nodos (se reportan)	0 y 0; las recepciones entre nodos se miden, no se prohíben
T06	Calidad del triaje	Conjunto etiquetado con verdad de referencia, por subgrupos.	*Recall* y precisión de P1; exactitud global (por separado)	*Recall* de P1 ≥ 95 %

ID	Qué verifica	Escenario (parámetros propuestos)	Métrica	Meta sugerida
T07	Deduplicación semántica	1 000 reportes en 80 incidentes más 20 pares adversariales.	Precisión y *recall* por pares; **tasa de falsa fusión**	Falsa fusión ≤ 1 %
T08	Rumor frente a corroboración	1 000 copias desde pocas fuentes frente a 5 fuentes independientes.	Confianza calculada	Confianza(rumor) < Confianza(corroborado)
T09	Caída de nodo	`kill -9` del nodo en pleno procesamiento.	Reportes con ACK perdidos; tiempo de recuperación (RTO)	0 perdidos; RTO a definir
T10	Partición y reconciliación	Dos nodos aislados 10 min, cada uno con reportes distintos; luego se reconectan.	Convergencia (huella de las proyecciones); tiempo	Huellas idénticas
T11	Cola bajo ráfaga	10 000 reportes en poco tiempo.	Latencia de P1; antigüedad máxima de P4 atendida	P1 no se degrada; sin inanición
T12	Degradación sin modelo	Modelo caído, lento o con `refusal`.	Calidad con y sin modelo; modo declarado	Degradación elegante, sin pérdida de reportes
T13	Seguridad de los agentes	Textos con instrucciones maliciosas; solicitudes sin datos suficientes; salidas fuera de esquema.	Acciones, recursos o ubicaciones inventados; salidas inválidas aceptadas	0
T14	Carga cognitiva del tablero	Lista de 1 000 reportes y 80 incidentes; revisión con personas.	Elementos a revisar para cubrir los P1; P1 fuera de la vista inicial	0 P1 ocultos; P1 a un clic
T15	Recorrido completo reproducible	Los ocho pasos con un solo comando y una semilla fija.	Huella de las proyecciones entre ejecuciones	Igual con la misma semilla
T16	Canal del despachador	Informes de radio, teléfono o en persona por `POST /api/dispatch`, con tres nodos reales, `kill -9` y réplica.	Aceptado, conservado y mostrado con su fuente; reintento sin duplicar; el mismo texto de varios orígenes cuenta como una fuente	Ningún informe aceptado se pierde ni se esconde; el reinicio no mata al nodo; un rumor no se amplifica

Resultados medidos en B3 (datos sintéticos; umbrales calibrados con unas semillas y cifras de otras generadas *después de congelar el código*; todo en `docs/resultados/b3.md`):

Tabla 25. *T06–T08 tal como se midieron (no tal como se esperaba).*

Prueba	Resultado	¿Cumple?
T06	Recall de P1 por reglas **78,5 %**; precisión de P1 100 %. El 12,8 % de los reportes P1 no trae información que lo delate: por reglas el techo es ≈ 87 %. Contando la revisión humana como acierto: 87,7 %, **escalando el 26 % de todos los reportes** (sólo el 23 % de los escalados era de verdad P1 o P2).	**No.** La brecha de la paráfrasis es la que el modelo debería cerrar, y no está medida.
T07	Con el θ calibrado que usa el nodo, falsa fusión **0,68 % en conjunto pero hasta 2,9 % en una de cinco semillas nunca vistas** (1,7 % en conjunto con los umbrales iniciales del PDF, hasta 6,3 % por semilla), con **recall de pares 0,4 %**. Sin texto agrupa 0 % y sugiere 15,7 % con 97 % de precisión.	**Sólo en conjunto y trivialmente**: casi no agrupa y una semilla de cinco lo incumple.

Prueba	Resultado	¿Cumple?
T08	Confianza 0,20 (1 000 copias de 3 fuentes) frente a 0,99 (5 fuentes independientes) sobre grupos dados.	**Sí, sobre grupos dados**; de punta a punta las 5 fuentes con palabras propias no se agrupan.

9.3 Métricas

Tabla 26. *Métricas principales y su razón de ser.*

Métrica	Definición	Por qué importa
***Recall* de P1**	Fracción de incidentes realmente críticos que el sistema clasifica como P1.	Un crítico omitido cuesta vidas; se reporta aparte de la exactitud global.
Tasa de falsa fusión	Pares de reportes de incidentes distintos que el sistema agrupó como el mismo.	El enunciado prohíbe «fusionar casos distintos indebidamente».
Razón de entrega	Reportes con ACK entre reportes creados, en un tiempo dado.	Mide el Reto 2 de punta a punta.
Bytes por reporte confirmado	Bytes de ida, de vuelta y de reintentos hasta el ACK.	Costo real para un teléfono con poca batería.
Latencia p50/p95 de P1	Tiempo desde la creación hasta que el reporte está en la cola del nodo.	La cola de espera de un crítico es lo que importa.
RTO y reportes perdidos	Tiempo hasta recuperar la función tras `kill -9` y número de reportes con ACK perdidos.	Mide el Reto 5.
Convergencia	Tiempo hasta que dos nodos reconectados tienen proyecciones idénticas.	Mide la sincronización.
Cobertura de evidencia	Fracción de salidas de agentes cuyas citas son válidas.	Mide las invenciones; objetivo 100 % tras validar.
Antigüedad máxima atendida	Edad del reporte más viejo servido por prioridad.	Detecta la inanición de casos leves.

Resultados de B4–B5 (T01 y T09–T15)

Medidos contra *procesos reales* del nodo por HTTP, con datos sintéticos del mismo autor del agente; el detalle, las cifras completas y los límites están en `docs/resultados/b5.md` y las decisiones tomadas por el camino en `docs/resultados/b5-decisiones.md`.

Tabla 27. *Resultados de B4–B5, con lo que cada cifra sí y no demuestra.*

Prueba	Resultado medido	Lectura honesta
T01	6 golpes de `kill -9` a un proceso que encola SOS: 0 pérdidas de lo dado por guardado, sin huecos, todas las tramas verifican.	Sólo ve la muerte del *proceso*; la atomicidad se prueba aparte con un fallo inyectado (la mutación «sin transacción» sobrevivía a T01). No prueba cortes de energía.
T09	`kill -9` con 26 reportes en ráfaga: A acusó 8 y los otros 18 conmutaron a B; todo lo acusado estaba al reiniciar; RTO 2,3–3,7 s.	RTO con `dart run` (JIT), una sola máquina.
T10	Huellas idénticas tras reconectar (3,1–3,6 s), incluso con decisiones contradictorias tomadas a la vez en cada nodo: ambos «rechazada + discrepancia».	Partición de *configuración* de 5–20 s, no de 10 min ni de red.
T11	10 000 reportes: de 515,6 s a ≈ 10 s; ACK p95 de 1 742 a 32 ms; P1 visible a los 2,6 s.	Lo corrigió quitar de la ruta del ACK un trabajo que crecía con el número de reportes; nunca se midió con teléfonos reales.

Prueba	Resultado medido	Lectura honesta
T12	Modelo colgado, con 500 o con basura: ACK p95 16–17 ms frente a 9 ms sin modelo; todas acusadas; cada respaldo contado por motivo. Reparto y relevo entre dos nodos. Ráfaga de 5 000 tramas con el modelo colgado y 1 024 descriptores: 0 fallidas.	El modelo es aquí un servidor **falso** a propósito (no se puede colgar la API real); la API real se midió aparte (T06 y T13 con modelo, abajo). Sin un tope de consultas simultáneas el nodo *moría* en esa ráfaga (se encontró al final y se corrigió); con el tope, la mayoría de los reportes con texto se quedan sólo con reglas.
T13	0 acciones o recursos inventados con textos de ataque, recursos falsos y salidas fuera de esquema; 0 aprobaciones automáticas.	La plantilla no usa modelo: lo que se prueba es la puerta de validación, que es la misma para cualquier proponente futuro.
T14	0 P1 fuera de la vista inicial en 5 semillas nuevas; P1 a 0 clics.	**No** es la prueba con coordinadores. La mejora (≈ 996 → ≈ 184 elementos) viene de ordenar, no de agrupar (≈ 183); con agrupamiento perfecto serían ≈ 20.
T15	Misma semilla ⇒ misma huella en cada par de ejecuciones (`bash tool/demo.sh 42`).	La huella cubre cola, incidentes, recomendaciones y aprobaciones; no relojes ni quién decidió.

Resultados con el modelo de lenguaje real (T06 y T13 con modelo)

Contra la API real, con datos sintéticos y **una sola corrida**: los mismos 750 reportes con texto (semillas nuevas 601–605) con reglas y con reglas + modelo, `effort low`. Gasto total ≈ 5,4 USD. Detalle, lectura y límites: `docs/resultados/modelo-real.md`.

Tabla 28. *T06 y T13 con el modelo real. El modelo nunca bajó una urgencia (piso por reglas) y sólo sube con una cita literal verificada.*

	Sólo reglas	+ Sonnet 5.5	+ Opus 5.5
Recall de P1 (con texto)	85,4 %	**96,0 %**	**98,5 %**
Falsos P1 (todos eran P2)	0	10	24
Precisión de P1	100 %	95,0 %	89,0 %
Latencia p95 · llamadas > 8 s	—	4,4 s · 0 de 749	5,6 s · 0 de 749
Coste por reporte	0	0,0024 USD	0,0048 USD
T13 con modelo (textos con instrucciones maliciosas)	—	12 de 12	12 de 12

Lo que no demuestra. Datos sintéticos del mismo autor que las reglas; una corrida (estos modelos no permiten fijar el muestreo); sólo reportes con texto, y ≈ 38 % de los P1 son SOS sin texto que no pasan por el modelo, así que el **criterio de T06 completo sigue sin cumplirse** (≈ 85 % global, estimado). Es un intercambio: más P1 capturados a cambio de más reportes P2 subidos un nivel. No hay prueba con reportes reales ni con coordinadores, y T12 sigue con un servidor falso.

Resultados del Reto 4 con un juez de modelo (T07 y T08 con modelo)

Contra la API real (Sonnet 5.5, `effort low`), una sola corrida, 963 consultas con semillas nuevas 701–705 y un criterio de adopción **commiteado antes de gastar**. El juez recibe cada reporte nuevo y hasta 6 candidatos anteriores que ya pasaron las reglas duras (ventana, tipo, distancia), y su respuesta se valida entera o se descarta. Detalle y protocolo: `docs/resultados/reto4-juez.md`. Gasto ≈ 6,2 USD.

Tabla 29. *Reto 4 con el modelo real. Aplicado el criterio fijado de antemano:* ***no se ofrece en el nodo*** *(falla la falsa fusión, el recall absoluto y las 5 fuentes completas).*

	Sólo reglas	Reglas + modelo
Recall de pares	0,81 %	**4,43 %**
Falsa fusión (criterio ≤ 1 %)	1,22 %	**2,21 %**
Grupos mezclados	1	6

	Sólo reglas	Reglas + modelo
Precisión de las uniones que hace el modelo	—	97,7 % (216 de 221)
T08: corroboración de 5 fuentes (grupos · fuentes del mayor · confianza)	5 · 1 · 0,60	2 · 4 · 0,97
T08: rumor de 12 copias (fuentes · confianza)	1 · 0,20	1 · 0,20
Fallos · llamadas > 8 s	—	5 de 963 · 1 de 958

Por qué falló y qué no demuestra. Los 5 *same* equivocados son gemelos adversariales del generador (misma calle, mismo vocabulario, casi a la vez) y los cinco tienen confianza exactamente 0,80, el umbral; cinco uniones malas bastan para casi duplicar la falsa fusión. Que un umbral de 0,85 lo arreglaría es una *hipótesis* sacada de estos mismos datos (descartaría casi la mitad de las uniones buenas) y exige semillas nuevas. Los textos de T08 son fijos: un caso, no cinco. Con 24 sucesos por conjunto el intervalo de la falsa fusión es ancho; las reglas solas ya pasan del 1 % en estas semillas. Datos sintéticos del mismo autor; una corrida. **No hay ninguna llamada al juez desde el nodo.**

Resultados del canal del despachador (T16)

Con tres nodos *reales* (`tool/dispatch_check.sh`) y el tablero compilado abierto en Chrome. El detalle, las decisiones y los límites están en `docs/resultados/despachador.md`.

Tabla 30. *Resultados de T16, con lo que sí y no demuestra.*

Prueba	Resultado	Límite
T16	Un informe se acepta, pasa por los agentes y aparece como incidente con su fuente (`radio`), medio, origen y quién lo registró, sin ubicación. Un reintento no escribe nada; el mismo id con otro contenido es 409; lo que no es JSON es 415, lo roto 400 y lo enorme 413, sin escribir nada. `kill -9` y reinicio: el nodo arranca y lo conserva (un `report.received` sin `frame_seq` lo tumbaba antes de esta prueba). La réplica lo lleva a otro nodo. Tres informes con el mismo texto y orígenes distintos son **una** fuente, confianza 0,2.	No se probó con despachadores ni con radio real. **No resuelve el Reto 4**: otros orígenes con otras palabras no se agrupan. La credibilidad 0,2 es un supuesto. Sin ubicación, el agente 6 no recomienda.

9.4 Informe de resultados (plantilla del entregable 6)

1. **Escenarios ejecutados:** archivos YAML con su semilla, versión del código y entorno.
2. **Resultados:** tablas y curvas de las métricas anteriores, con la comparación de modo reglas y modo modelo.
3. **Errores observados:** fallas reales encontradas durante las pruebas y cómo se resolvieron.
4. **Limitaciones:** qué no se probó (canales reales, escala real, dispositivos iOS) y qué se simplificó.
5. **Reproducción:** los comandos exactos para volver a ejecutar cada escenario.

Documentar con honestidad las limitaciones y los errores observados es parte de lo que el enunciado pide y es lo que permite al jurado auditar la solución.

10 Plan de desarrollo

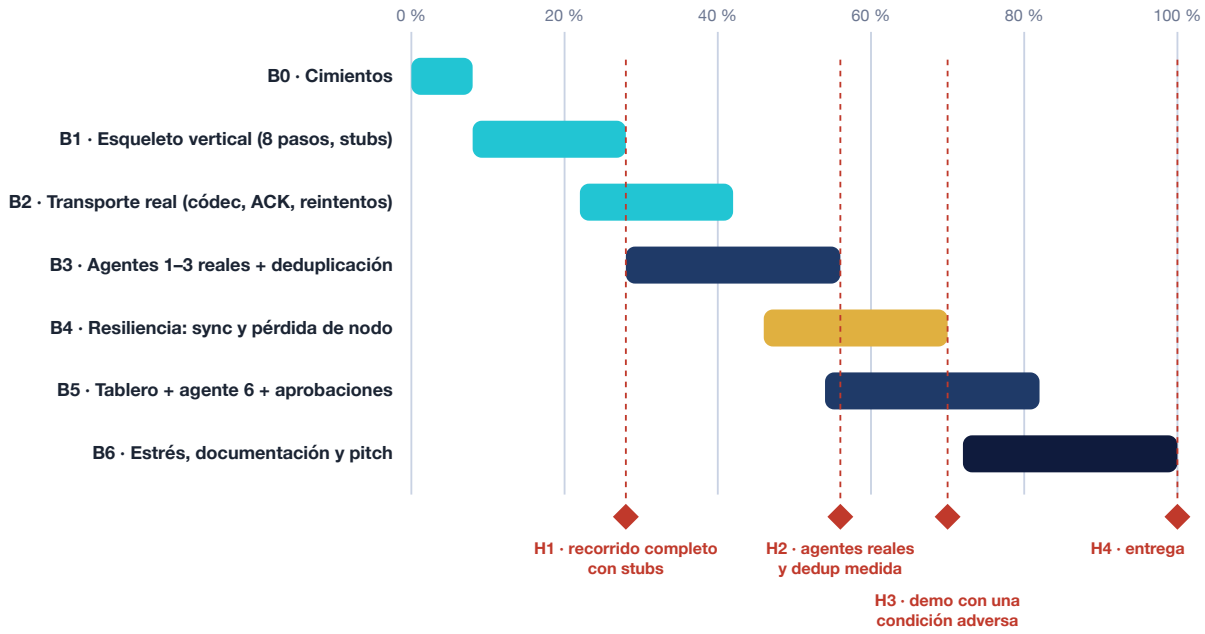
El plan está pensado para una restricción que el enunciado no resuelve: no sabemos cuánto tiempo hay. Por eso se expresa en **bloques con criterio de terminado** y porcentajes del tiempo disponible, y está ordenado para que siempre exista un recorrido completo funcionando desde el primer tercio del tiempo, aunque sea con piezas simples.

10.1 Estructura del repositorio

Un monorepo con *pub workspaces* de Dart. Los paquetes compartidos son Dart puro, sin Flutter. Los bloques B0, B1 y B2 ya están implementados (planes en `docs/superpowers/plans/`, resultados en `docs/resultados/`). Las dos apuestas de la pila se validaron antes de construir: SQLite durable ante `kill -9` en el nodo, y el espacio de trabajo con la app Flutter, el paquete puro y Flutter Web.

```
hackathon-asi/
├── HackathonExpoASI2026.pdf     enunciado oficial (no se modifica)
├── README.md
├── pubspec.yaml                 workspace de Dart (se crea en B0)
├── apps/
│   ├── mobile/                  app ciudadana (Flutter)
│   ├── node/                    nodo superviviente en Dart: API, log, sincronización, agentes
│   └── dashboard/               tablero del coordinador (Flutter Web)
├── packages/
│   ├── protocol/                códec binario, esquemas y vectores de prueba
│   ├── agents/                  seis agentes, validadores, puerto LLM y respaldos
│   └── sim/                     generador sintético, simulador de canal, inyector de fallos, métricas
├── scenarios/                   escenarios YAML reproducibles (semilla fija)
└── docs/
    ├── SARD-arquitectura-app-movil.pdf   este documento
    ├── adr/                     decisiones de arquitectura, una por archivo
    ├── agentes/                 fichas de agentes (entregable 4)
    ├── protocolo.md             especificación SARD-WIRE (entregable 5)
    ├── resultados/              informes de pruebas (entregable 6)
    └── src/                     fuentes y generador de este PDF
```

10.2 Hoja de ruta por bloques



Eje horizontal: porcentaje del tiempo disponible (escalar al plazo real del equipo).

Figura 10. *Bloques de trabajo con solapamiento deliberado e hitos de integración (H1–H4). Las barras indican esfuerzo relativo, no fechas.*

Tabla 31. *Bloques, contenido y criterio de terminado.*

Bloque	Qué se construye	Criterio de terminado
B0 · Cimientos	Entorno verificado; las dos pruebas técnicas; monorepo con *pub workspaces*; paquete `protocol` (Dart puro) con el códec y los vectores de prueba; primeras decisiones de arquitectura.	`dart test -p vm,chrome` pasa; el códec reproduce el vector de §7.2.
B1 · Esqueleto vertical	App con SOS y bandeja; transporte HTTP simple; nodo con log y agentes «stub»; tablero con lista; script que mata un nodo.	**H1:** los ocho pasos corren con datos sintéticos de punta a punta, aunque con piezas simples.
B2 · Transporte real	Códec binario en la app, ACK selectivo, reintentos con variación y ventanas; proxy de canal con pérdida, duplicados y retardo.	T02–T05 en verde.
B3 · Agentes y deduplicación	Agentes 1, 2 y 3 reales (reglas y modelo); generador sintético con verdad de referencia; puerto LLM con respaldo.	**H2:** T06–T08 medidos y reportados.
B4 · Resiliencia	Anti-entropía, dueño por *rendezvous*, agente 5, reconstrucción de proyecciones, inyector de fallos.	**H3:** demostración con una condición adversa (T09, T10).
B5 · Tablero y coordinación	Lista corta con contadores, evidencia y contradicciones, agente 6, aprobación, frescura.	T14; la aprobación queda registrada.
B6 · Estrés y entrega	T11–T13 y T15 finales, informe de resultados, documentación, guion y video de respaldo.	**H4:** los siete entregables completos.

10.3 Si el tiempo es corto: qué mantener y qué simplificar

El núcleo no negociable es el recorrido de ocho pasos con **una condición adversa** demostrada (entregable 7). El resto puede reducirse sin invalidar el diseño, siempre que las simplificaciones queden declaradas como limitaciones.

Mantener

- SOS en un toque, bandeja durable, estados visibles.
- ACK y reintentos con variación.
- Log de eventos y `kill -9` con recuperación.
- Triaje por reglas con evidencia.
- Tablero con lista y «ver todo».
- T01, T03 y T09 reproducibles.

Simplificar

- Códec: empezar con JSON compacto detrás de la misma interfaz y cambiarlo después.
- Sincronización: replicación por sondeo en vez de anti-entropía completa.
- Correlación: puntaje determinista sin modelo.
- Modelo de lenguaje: sólo en el triaje.

Posponer

- Canales reales (BLE, SMS).
- Tramas delta y confirmación de segundo nivel.
- Modelo de lenguaje local.
- Mapa en el tablero.
- Modo «sólo SOS» por debajo del 5 %.

10.4 Roles

Cuatro frentes de trabajo. **Este proyecto lo desarrolla una sola persona**, así que los frentes no son un reparto sino un orden: móvil y transporte, nodo, agentes, tablero y relato. Con una persona, la columna «Mantener» de §10.3 es el plan, y todo lo demás se declara como limitación.

Tabla 32. *Reparto sugerido. El contrato entre frentes es el paquete de protocolo y los esquemas de evento.*

Frente	Responsabilidad principal	Bloques donde pesa
Móvil y transporte	App, planificador de envío, códec en el dispositivo, pruebas de energía en un teléfono real.	B1, B2
Nodo y resiliencia	Log, sincronización, agentes 4 y 5, despliegue de nodos, inyector de fallos.	B1, B4
Agentes y datos	Agentes 1, 2, 3 y 6, puerto LLM, generador sintético, métricas de calidad.	B3, B5
Tablero, pruebas y relato	Tablero, orquestador de escenarios, informe de resultados, pitch y documentación.	B5, B6

10.5 Riesgos del proyecto

Tabla 33. *Riesgos y mitigaciones.*

Riesgo	Mitigación
Querer cubrir todo y no tener nada completo	Recorrido vertical primero (H1 hacia el 28 % del tiempo); bloques con criterio de terminado.
El modelo de lenguaje falla o no hay Internet en la demostración	Modo de reglas y respuestas grabadas; el puerto lo permite desde el diseño.
Falla en vivo de la demostración	Escenario con semilla y un solo comando; video de respaldo; datos locales.
Problemas del dispositivo real (permisos, batería, fabricante)	Probar en el teléfono real desde B1, no sólo en emulador.
Calibración lenta de la deduplicación	Conjunto sintético con verdad de referencia desde B3; umbrales conservadores que priorizan no fusionar.
Sincronización más compleja de lo previsto	Empezar con replicación por sondeo y evolucionar a anti-entropía; los eventos inmutables lo permiten.

Riesgo	Mitigación
Sobreingeniería del códec binario	Vectores de prueba y JSON compacto como paso intermedio.

10.6 Guion de la demostración

El entregable 7 pide una demostración reproducible bajo al menos una condición adversa. Este guion encadena varias, y cada paso apunta a una prueba que lo respalda. Se adapta a la duración real del pitch.

Tabla 34. *Guion en siete pasos con la prueba que respalda cada uno.*

N.º	Qué se ve	Qué demuestra
1	Un SOS desde el teléfono en modo avión; se mata la app y se reabre: el reporte sigue «Guardado en tu teléfono».	Captura resiliente (T01).
2	El proxy aplica 60 % de pérdida y ventanas de 3 s; el SOS llega; se muestran los 33 B, el ACK y los reintentos con variación.	Comunicación en ráfagas (T03, T04).
3	Entran 200 reportes sintéticos; el tablero los agrupa en incidentes y pone los P1 al frente; un rumor copiado frente a cinco fuentes independientes.	Triaje, deduplicación y corroboración (T06–T08).
4	**Condición adversa:** `kill -9` del Nodo A en vivo; el tablero del Nodo B sigue y muestra «vista parcial»; la app reintenta contra B.	Continuidad operativa (T09).
5	Se reinicia A; sus proyecciones se reconstruyen y convergen; las huellas de A y B coinciden.	Sincronización y recuperación (T10).
6	El coordinador aprueba una recomendación; se muestra que los agentes no tienen herramienta de despacho y el registro de auditoría.	Humano en el lazo y uso responsable (T13).
7	Tabla de métricas y limitaciones declaradas.	Validación experimental (entregable 6).

En el repositorio. `bash tool/demo.sh 42` ejecuta la condición adversa de los pasos 4 y 5 con dos nodos reales (partición, `kill -9` con ráfaga en curso, conmutación, reinicio con su RTO, reconexión) más las *decisiones contradictorias* de dos coordinadores, y lo repite para comprobar que la misma semilla da la misma huella; `PART=20` alarga la partición. El guion de pitch con tiempos está en `docs/pitch/guion.md`. Los pasos 1–2 los cubren T01 y el recorrido automático del dispositivo; el 3, el generador y `packages/sim/bin/t14.dart`.

10.7 Próximos pasos inmediatos

1. **Confirmar el plazo disponible**: es el dato que falta para dimensionar el recorte de §10.3 para los bloques B3 a B6.
2. **B3 hecho y medido (H2):** agentes 1, 2 y 3, generador sintético con verdad de referencia, puerto del modelo con respaldo y T06–T08 medidos. **Dos de los tres criterios no se cumplen** (T06 78,5 %; T07 sólo en conjunto y trivialmente: una de cinco semillas nuevas supera el 1 %) y se reportan así, con su diagnóstico, en `docs/resultados/b3.md`. Para cerrar la brecha hacía falta un modelo real: se midió después, el 9 de noviembre (docs/resultados/modelo-real.md: el recall de P1 en reportes con texto pasa del 85,4 % al 96,0 %, pero el criterio de T06 completo sigue sin cumplirse) y, para la deduplicación, una señal que separe sucesos vecinos casi idénticos. **Siguiente:** B4 · resiliencia (anti-entropía, dueño por *rendezvous*, agente 5).
3. **B4 y B5 hechos y medidos** (`docs/resultados/b5.md`): la app captura personas y frase; el nodo calcula incidentes con recomendación del agente 6 y aprobación registrada; el tablero los muestra con contadores honestos y «vista parcial»; el trabajo caro se reparte por *rendezvous* con relevo; la ráfaga de 10 000 pasó de 516 s a ≈ 10 s. **Lo que no se logró o no se probó:** el modelo de lenguaje no se había consultado de verdad al

cerrar B5 (después se midió, ver docs/resultados/modelo-real.md; T12 y la ráfaga siguen con un servidor falso); el agrupamiento sigue sin fusionar casi nada, así que la ventaja del tablero es de orden y no de agrupación; no hubo prueba con coordinadores; la partición se probó 5–20 s, no 10 min; quien aprueba se protege con un código compartido (opcional, `--coordinator-token`) pero no se identifica, el código viaja sin cifrar y la clave de integridad es compartida.

4. **Probar en hardware real:** B2 se verificó en el emulador de Android y el simulador de iOS (ubicación y batería simuladas); falta un teléfono real, un servicio en primer plano de Android y, si se quiere, un canal de baja capacidad.
5. **Hallazgo de B2 sin resolver:** en ventanas de 3 s cada 60 s, el 100 % de los SOS sale en la primera ventana *sólo si el sistema avisa bien de cuándo hay red*; si dice que hay red pero el nodo no responde (intentos de 5 s) baja al **20 %** (todos acaban saliendo después). Mejoras a medir: un plazo de intento corto justo tras volver la señal y reintentar enseguida mientras el enlace responde.

A Glosario

Término	Significado en este documento
ACK selectivo	Confirmación de recepción de una trama concreta, con un mapa de bits de las 32 tramas anteriores que el mismo nodo también guarda.
Anti-entropía	Mecanismo por el que dos nodos comparan lo que tienen e intercambian sólo lo que le falta a cada uno.
Append-only	Almacén donde sólo se añaden registros; nunca se modifican ni se borran.
Envejecimiento (*aging*)	Aumento gradual de prioridad con la espera, para que un caso leve no quede relegado indefinidamente.
***Backoff* con variación**	Espera creciente entre reintentos con un componente aleatorio, para que muchos dispositivos no reintenten a la vez.
Event sourcing	Guardar la secuencia de eventos como fuente de verdad y derivar las vistas de ella.
Geohash	Código de texto que identifica una celda geográfica; las celdas vecinas comparten prefijo.
HLC	Reloj lógico híbrido: ordena eventos de forma causal sin depender de que los relojes de los nodos coincidan.
Idempotencia	Propiedad de una operación que, repetida varias veces, deja el mismo resultado que una vez.
MTU	Tamaño máximo de un paquete que un canal puede transportar sin fragmentar.
Noisy-OR	Regla para combinar probabilidades de fuentes independientes: $1 - \Pi(1 - p_i)$.
Proyección	Vista derivada del log (cola priorizada, grupos de incidentes) que puede reconstruirse desde cero.
Rendezvous hashing	Forma de asignar un trabajo a un nodo vivo de modo que, si cae, otro lo hereda de forma determinista.
Store-and-forward	Guardar un mensaje localmente y reenviarlo cuando haya una oportunidad de transmitir.
TTL	Tiempo de vida tras el cual cambia el tratamiento de un mensaje.
Vector de versiones	Resumen, por nodo de origen, de hasta qué contador se tienen todos los eventos.
WAL	Registro previo de escritura de SQLite que permite recuperación consistente tras un corte.

B Decisiones de arquitectura (resumen)

Resumen de las decisiones principales en formato de registro. Cada una se guardará como un archivo independiente en `docs/adr/` al iniciar la implementación.

ID	Decisión	Alternativa descartada	Consecuencia principal
01	Log de eventos *append-only* con vistas derivadas	Base mutable con réplicas	Recuperación y auditoría simples; el coste es reconstruir vistas.
02	Sin líder y consistencia eventual	Consenso (Raft)	Sigue funcionando con particiones; las vistas pueden diferir de forma temporal y se declaran.
03	Monorepo Dart con paquetes puros compartidos (`protocol`, `agents`, `sim`)	Varios repositorios y lenguajes	Menos fricción para una sola persona; los agentes quedan en Dart y no hay SDK oficial de Claude (se usa HTTP directo).
04	Trama binaria propia con vectores de prueba	JSON o Protocol Buffers	33 B frente a 239 B (JSON descriptivo) o 104 B (JSON compacto); hay que mantener el códec y sus pruebas.
05	Modelo de lenguaje tras un puerto con respaldo determinista	Dependencia directa de una API	Funciona sin Internet; hay que mantener las reglas.
06	Agentes sin herramienta de despacho	Autonomía limitada sólo por instrucciones	Límite por construcción; ninguna acción es automática.
07	Deduplicación semántica como vista reversible	Fusionar y borrar	Conserva la evidencia; exige más lógica de vistas.
08	Banco de pruebas como componente de primera clase	Pruebas manuales al final	Reproducibilidad y evidencia para el jurado; coste inicial de desarrollo.
09	Reporte progresivo por revisiones	Formulario único	Mínima interacción y sin duplicados de origen; hay que fundir campo a campo.
10	Desarrollo individual con corte de alcance en la columna «Mantener»	Intentar cubrir todos los retos	Se entrega el recorrido completo con piezas simples y limitaciones declaradas; la profundización queda como trabajo futuro.